重庆交通大学
CHONGQING JIAOTONG UNIVERSITY

本科毕业设计（论文）

题　　目：_____

_____

学　　院：信息科学与工程学院

专　　业：计算机科学与技术

学 生 姓 名：_____

学　　号：_____

指 导 教 师：_____

评 阅 教 师：_____

完 成 时 间：_____

重庆交通大学

CHONGQING JIAOTONG UNIVERSITY

本科毕业设计（论文）原创性声明

本人郑重声明：所提交的毕业设计（论文），是本人在导师指导下，独立进行研究工作所取得的成果。除文中已注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。对本文研究做出过重要贡献的个人和集体，均已在文中以明确方式标明。

本人完全意识到本声明的法律后果由本人承担。

作者签名（亲笔）：　　年　　月　　日

本科毕业设计（论文）版权使用授权书

本毕业设计（论文）作者完全了解学校有关保留、使用学位论文的规定，本科生在校攻读期间毕业设计（论文）工作的知识产权单位属重庆交通大学，同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅；本人授权重庆交通大学可以将毕业设计（论文）的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存、汇编毕业设计（论文）。

作者签名（亲笔）：　　年　　月　　日

导师签名（亲笔）：　　年　　月　　日

摘 要

近年来，随着国家绿色环保思想的大力宣传，信息技术、社会经济的发展，国内越来多的消费者们开始加入二手物品交易的市场中，消费者们将身边不常用但完好的物品投入到二手市场中进行交易，同时消费者们也可以在二手市场中淘到心仪性价比高的物品。这样的交易流转，让消费者们不仅是买家也是卖家。闲置的物品们也得以发挥其价值。根据 2019 年数据调查显示，2019 年我国二手行业交易规模达到 2596.9 亿元，同时同比增长率也直线上升。所有构建一个完善标准化的二手交易市场是大势所趋。

本文针对国内现有二手系统进行调查研究后，设计出一款 B/S 网页版的二手物品交易系统，主要由前台用户系统以及后台管理员系统组成。前端页面和其功能使用 Layui、Vue、JS、Ajax、jQuery 等进行实现，利用可视化图形界面展示，做到前端界面简洁明了，用户上手快。后端采用 SpringBoot+MyBatis 框架，利用 java 语言进行书写。数据库使用关系型数据库 MYSQL 对数据进行存储与操作。

关 键 词： 二 手 交 易 ； SpringBoot+MyBatis ； Layui ； B/S 结 构

The second-hand goods trading system -- the design and implementation of the seller module

Abstract

In recent years, with the vigorous publicity of the national green environmental protection idea, the development of information technology and social economy, more and more domestic consumers begin to join the second-hand goods trading market. Consumers put the rarely used but intact goods into the second-hand market for trading. At the same time, consumers can also find the goods with high cost performance in the second-hand market. This kind of transaction circulation makes consumers not only buyers but also sellers. Idle items are also able to play their value. According to the data survey in 2019, the transaction scale of China's second-hand industry will reach 259.69 billion yuan in 2019, and the year-on-year growth rate will also rise sharply. Therefore, it is the general trend to build a perfect and standardized second-hand trading market.

This paper designs a B / s web page version of second-hand goods trading system, which is mainly composed of front-end user system and back-end administrator system. The front-end page and its functions are implemented with layui, Vue, JS, AJAX, jQuery, etc., and displayed with visual graphical interface, so that the front-end interface is simple and clear, and users can get started quickly. The back-end uses springboot + mybatis framework and uses Java language to write. The database uses the relational database mysql to store and operate the data.

Key Words： Second hand transactions; SpringBoot+MyBatis； Layui； B / S structure

目　录

摘　要 II
Abstract III
第 1 章 绪论 1
1.1 项目背景及意义 1
1.2 国内外现状 3
1.2.1 国内现状 3
1.2.2 国外现状 4
1.3 目前存在的问题 6
1.4 本文的主要内容 6
第 2 章 系统开发相关理论基础 8
2.1 系统开发语言 java 8
2.2 系统开发平台 9
2.3 系统框架 9
2.4 系统架构 11
2.5 相关技术简介 12
第 3 章 系统可行性与需求分析 14
3.1 系统基本情况描述 14
3.2 系统可行性分析 14
3.3 系统功能需求分析 16
3.3.1 系统功能概述 16
3.3.2 系统业务分析 16
3.3.3 用户 UML 用例图 21
第 4 章 系统总体设计 23
4.1 系统模块设计 23
4.2 系统流程设计 25
4.3 系统数据库设计 26
4.3.1 数据库概念模型 27
4.3.2 数据库逻辑模型 32
4.3.3 数据库物理模型 33
第 5 章 系统详细设计 37
5.1 买家模块设计 37

5.2 后台模块设计 39
5.3 卖家模块设计 41
6 卖家模块的实现 44
6.1 用户管理 44
6.1.1 修改基本信息 44
6.1.2 修改密码 48
6.1.3 修改邮箱 50
6.2 商品管理 53
6.2.1 上架商品 53
6.2.2 编辑商品 57
6.2.3 下架商品 58
6.2.4 查看商品 59
6.3 订单管理 63
6.3.1 查看订单 63
6.3.2 卖家发货 64
6.4 退货退款与上诉 65
第 7 章 系统测试 68
7.1 系统测试目的与方法 68
7.2 系统测试用例 69
7.3 测试总结 71
第 8 章 结论与展望 72
致 谢 73
参 考 文 献 74

第 1 章 绪论

1.1 项目背景及意义

2020年是我国脱贫攻坚战的收官之年，随着信息技术的发展，网购规模迅速扩大，网购销售渠道逐渐成熟，电商高速扩张，价格优势彰显，售后服务周到，借助快递行业的急速发展，网购已成为一大购物主流。随之而来的是闲置经济的崛起，二手电商主要的交易形式包含四种，分别是二手买卖、置换、回收和捐赠，所有类型的消费品都可以被用来买卖，最先冲出行业的是二手车、二手房，紧接着二手数码、二手服装、二手奢侈品、二手图书、二手家具等等都展示在人们面前。据数据显示，2019年中国闲置市场规模达8834亿元，同比增长19%，预计2020年市场规模将超百亿，达到20409亿元[1]。

图1.1 2014-2020年中国闲置市场规模及增速预测

国务院曾表示，共享经济也是众创经济[2]，互联网的深入发展、日益普及，为人民群众工作环境提供了广阔的空间与机会。论其本质，共享经济是整合了线下的闲散物品或服务者，让他们以较低的价格提工产品或服务。同时各个地方政策也在大力推行节约、绿色的基本国策，上海出台的严格的垃圾分类回收政策就曾在网络上刮起一阵“侬是什么垃圾”的奇幻风，当人们面临要在丢弃物上花费时间与成本来进行处理时，为何不把一些有用物品上架到二手平台成为一大解决方式，这样不仅节约了垃圾分类的时间，同时也将“无价值”闲置换来一些价值。不仅仅是该政策，很多类似的政策其实都在无意识中促进二手市场的发展，帮助二手交易平台对闲置武平进行分类回收的成本，使得现在来说比较流行的C2B2C方式的前端成本大幅降低[3]。所以，随着国家政策在绿色环保方面的推行下，二手市场的发展将愈发强大。

二手平台主要采用有C2C、C2B2C两种形式。最早的可以追溯到2002年的孔夫子旧书网，他们采用的仅仅是将交易信息线上化，其余的购物流程依旧在线下进行，后来随着信息化技术、互联网、快递行业的发展，58同城、赶集网、闲鱼、转转、拍拍的等二手信息平台逐渐进入人们的眼帘。C2C的运行模式多是与综合品类互相绑定，在进行精细化运营提高交易转化率，例如淘宝天猫旗下的闲鱼。C2B2C的运行模式则是从垂直平类入手，建成壁垒后仔横向扩张品类，例如京东旗下的爱回收，专注于对3C数码产品的运营。同时也有从C2C模式向着C2B2C模式进行转换的平台，例如58同城下部署的转转，产品从综合类逐渐过渡到3C电子产业。

因此，为了不断贯彻实行绿色可持续发展观念，践行节约、节俭美德，通过二手交易系统，实现买卖双方双赢，以信息化的方式来构建时代二手交易市场势在必行。本次研究的题目二手物品网上交易系统——卖家模块的设计与实现便是争对这种需求中卖家模块所进行的研究。

1.2 国内外二手系统

1.2.1 国内二手系统

目前，我国二手电商行业初步形成了以闲鱼、转转为首，爱回收、享物说、5miles、铁甲二手机、胡斯后报、红布林、找靓机、花粉儿、只二、心上、胖虎、孔夫子旧书网、阅邻等的综合性市场。

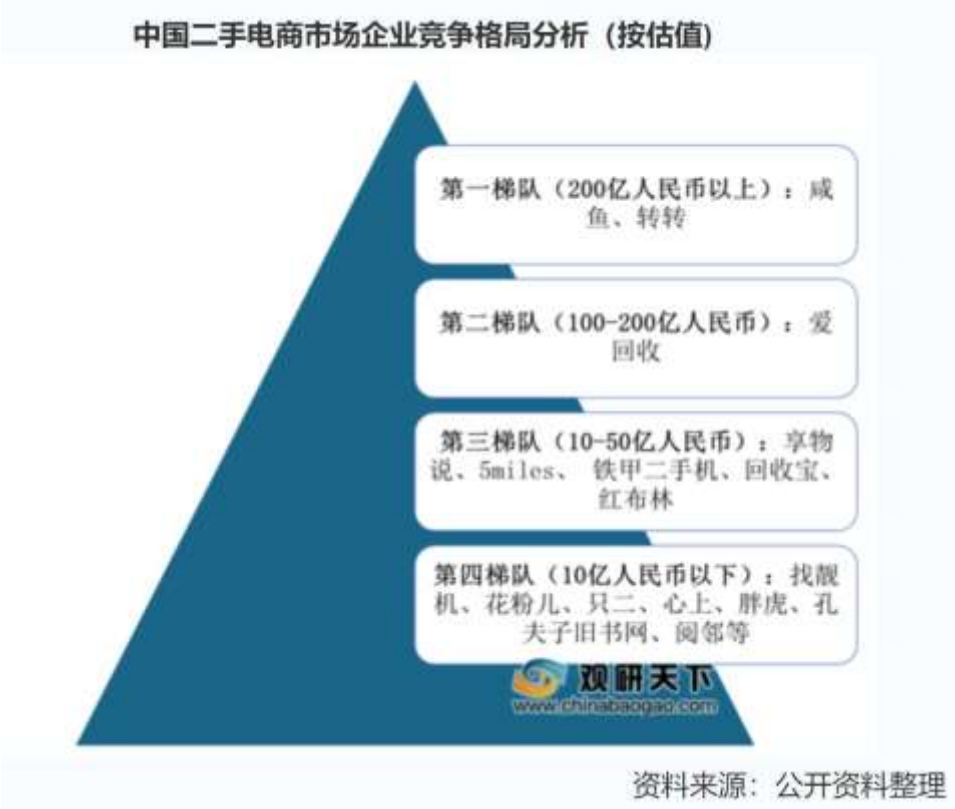

图1.2 中国二手电商市场企业竞争格局分析

根据图 1.2 数据显示，综合型的平台闲鱼、转转估值达 200 亿人名币以上，占据二手电商市场 90.0%的市场份额，渗透率分别达 72.9%和 33.1%。以下以闲鱼、爱回收为例。

闲鱼主要采用的是 C2C 为主，加码 C2B2C 为补充的模式，重点放在了为个体用户提供服务，次要提供自营商品[4]。由用户自行发布、淘宝一键转卖、省心买三种交易方式，为保障交易顺利进行，由芝麻信息评估用户信息，并提供省心卖（估价、取

件、帮卖）以及无忧购（包真验货、包邮、包退、售后）等服务，总体来说是一个成熟的二手交易系统，拥有安全的、专业的交易机制，完善的信用交易制度，并且，在淘宝的引流加持下，闲鱼占据绝大部分市场无可厚非。

与之相对比的我们来看一看京东平台下的爱回收二手系统。爱回收主要采用的是 C2B2C 的模式，专注于 3C 数码商品的闲置售卖。经营模式则为自营产品占据平台，也因为 3C 数码产品的特殊性，爱回收建立了一套完整的有针对性的运营流程。闲置来源同样是由用户发布以及京东的一键转卖，平台将为用户提供线上评估邮寄回收、免费上门回收、线下门店回收的服务，利用京东小白信息以及专业质检商对闲置数码产品进行检查，爱回收所建立的标准化手机回收处理流程，国防级的 Blancco 数据安全以及线下门店都为爱回收在 3C 数码闲置行业打下重要的基础[5]。

不同的模式各有不同的侧重点以及各自的优势，为此所展现出来的用户新需求，行业发展新痛点也不同，对此将在 1.3 小节具体展开论述。

1.2.2 国外二手系统

在发达国家的二手平台市场中我们发现，无论是用户量还是活跃的融资动态，在发达程度比较高的国家二手交易平台发展也更加多种多样且成熟。以美国为例，最典型的美式二手交易始于 20 世纪 50-60 年代，当时的美国正处于二战后城市飞速扩张的时期，大部分人都进行了搬家活动，大型家具搬迁问题促进了二手市场的发展，一直到 21 世纪初，人们不仅仅使用报纸、告示牌进行发布、寻找二手交易信息，开始尝试利用网络进行二手交易，直到今日，美国已发展出多个具有鲜明特点的二手交易平台。

首先是 Letgo，这是一个不支持线上支付的二手交易平台，他所利用的是地理位置[6]。Letgo 是目前美国本土规模最大，增长最快的二手交易平台，其主要服务对象集中在美国本土、土耳其、加拿大、挪威、西班牙，在中国也有少量涉及。这与中国流行的线上交易有所不同，反而与中国前期孔夫子旧书网较为类似。Poshmark，一个专注于旧服饰个人买卖市场的社交性应用程序，与此同时加入符合大众潮流的品牌与物件，参照最受欢迎的用户影响力批发价进货，从而从中获利。Tradesy，专注于奢侈品二手交易。

在新冠疫情的影响下，美国二手交易市场得到了更大的发展。2020 年假期期间，由 Tradesy 美国二手在线奢侈品平台报告显示，二手高端鞋子、珠宝、手袋的销量急剧增加。同时由美国二手服装电商平台 ThredUp 在 2020 年发布的市场报告（图 1.3）中显示，预计未来五年内市场将增长到 640 亿美元，到了 2029 年，整个市场的总价值甚至有望达到 800 亿美元[7]。

图1.3　2009-2029年美国二手市场价值预测

最后，不仅仅是美国，韩国、新加坡、日本等国家也纷纷在二手行业发展迅速，所以越是在发展程度高的国家，人们的环保意识也会更加强烈，二手交易的闲置资源也将得到更好的利用，这样既满足了基本物质需求的保障，又可以通过二手交易平台赚钱、获得一些有收藏价值的物品。

1.3 目前存在的问题

针对二手交易平台 2020 年全年的消费舆情信息统计，在 856873 条信息中负面舆情信息占到了高达 61.63%的占比，这一项尖锐的数据告诉我们中国二手交易市场虽有万亿级的消费市场，但其中问题漏洞却占据了一大部分还未被解决。

（1）商品质量的问题。“柠檬”问题即“次品或二手品”问题[8]。信息不对称是目前行业中比较严重的问题，当卖家给出的信息不足以使买家全方位的了解该二手商品，或卖家为了好价故意隐藏信息等都将对双方造成信用危机，当信用危机堆积，市场就将难以运作。

（2）供求问题。二手市场产品不完整，如何找到合适的卖家与买家，如何将物品推送给正有所需的人，如何在众多信息中筛选出最合适的商品成为一大难题。

（3）信任问题。由于很大一部分二手交易平台缺乏对商品、用户的管理，导致市场商品鱼龙混杂，以次充好现象频出，用户难免会在交易中陷入骗局，出现交易风险，这样的情况影响了用户体验，也导致了平台信誉受损[9]。

所以需要平台加强对信息的管理，对产品采用更加精细的等级划分，在用户流量、征信系统、交易担保、物流等设施上加大投入。国家也因出台相关政策，保护平台、消费者权益，积极引导网民的走向，创造绿色和平的二手交易环境。

1.4 本文的主要内容

易购网上物品二手交易系统是由卖家、买家以及后台管理三大模块组成，本文的主要工作是卖家模块的设计与实现。系统采用 B/S 模式，以互联网为基础，服务器租用阿里云学生服务器，使用编程工具是 IntelliJ IDEA 2018.3.2 x64，采用 SpringBoot+Mybatis 框架技术设计实现，后台使用 Mysql 数据库处理数据，前端使用 vue+layui 进行装饰，使用 JavaScript 动态处理数据。

本文的各个章节的主要内容如下：

第一章，对项目背景、意义等进行阐述。

第二章，对系统开发相关理论基础进行阐述。

第三章，对系统可行性分析与需求分析。

第四章，对系统进行总体设计。

第五章，对系统进行详细设计。

第六章，对卖家模块进行设计。

第七章，对系统进行测试。

第 2 章 系统开发相关理论基础

2.1 系统开发语言 java

Java 语言，由 SUN 公司在 1995 年发行的新型面向对象类程序设计语言[10]，与 C++将为类似但由于其有所不同。Java 语言的初衷是为了实现语言非常小但却能生成紧凑代码，同时要求语言可跨平台，保持其中立，为此，Java 语言诞生了。java 拥有新的从程序设计平台，适用于 Internet 应用程序开发。到目前为止，Java 语言已成为软件开发革命性的一项技术。

Java 简单性。Java 继承了 C++的优点，删除了 C++中复杂的继承，没有头文件、指针运算、结构、联合、操作符重载、虚基类等等，可以说是一个纯净版的 C++语言。

Java 面向对象。Java 采用更简单的接口概念代替 C++中复杂的多继承关系，其语言灵活且强大，可读性强，可重用性好且可移植。

Java 分布性。Java 语言具有丰富的例程库，使其可通过 URL 打开并访问网络上的对象，且程序可多处使用，节省人力资源。

Java 编译和解释性。作为生成字节码的编译程序，其运行速度快于其他开发程序。

Java 稳健性与安全性。Java 可检查出其他语言必须运行才能发现的错误。其程序可以防范网络各种攻击，该特性使得很多公司都会选择 Java 进行开发。

Java 可移植性。不依赖平台保持中立，可运用于任何操作系统。

Java 高性能。先编译后解释，字节码可在运行时刻动态的翻译成对应的机器码。

Java 多线程与动态性。支持并发，实现实时的交互响应与行为。可自由添加新方法，且不影响客户端。

2.2 系统开发平台

IntelliJ IDEA 是 JetBrains 公司下的 Java 集成开发环境，2001 年 1 月发布 1.0 版本，至今已发展到 2021.1.1 版本。其支持几乎所有主流技术和框架，擅长于企业应用、移动应用和 Web 应用的开发，非常的人性化、智能化、可自我定制化，开发效率极高。该系统使用的是 IntelliJ IDEA 2018.3.2 x64，虽然版本比较古老，但其功能已足够。

IDEA 索引。支持快速索引，及时进行信息反馈，同时支持 ALT+ENTER 修复，使代码可以很快的完成。

IDEA 语言支持。IDEA 支持多种语言，且处理方式和谐统一。

IDEA 调试器。速度迅速且根据需求不同采用多种调试按钮，实时反馈调试结果。

IDEA 内存占用率小，使用软件运行迅速且不卡顿，就算是长时间使用也不会出现假死情况。

IDEA 强大的插件市场。虽然 IDEA 本身已提供了几乎所有类型的功能，但任然某些基本功能需要插件的帮助，这是 IDEA 所具备的插件市场为完成代码提供了很好的帮助，开发者也可以自行编写插件将其上传至插件市场为他人提供帮助。这样的共享式插件市场使得 IDEA 具有强大的可写性，激发了生产者的动力。

2.3 系统框架

（1）后端框架 SpringBoot+Mybatis

SpringBoot 由 Pivotal 团队为了简化 Spring 搭建初始项目时复杂的配置过程而开发出来的一款简单的框架[11]。程序员运用该框架方便、快捷的搭建部署项目，狂渐渐的兼容性、版本等都不需要考虑，添加新配置即可解决问题，SpringBoot 非常适合构建微服务。

MyBatis 由 Apache 所提供的开源项目 iBatis 所有衍生而来的基于 Java 持久层的框架，目前隶属于 GitHub。提供了 SQL Maps、Data Access Object（DAO）等，使用简单的 XML 或注解，对配置和原始映射进行服务。MyBatis 非常小巧、方便、高效、简单、直接、半自动化，与 SpringBoot 非常合适。

其中 SpringBoot+MyBatis 架构如下：

POJO（Plain Ordinary Java Object）：Java 对象

Mapper：数据访问层，MyBatis 操作数据库层，类似于 dao 层。

Service：业务逻辑层，业务处理和数据传递，将 Mapper 传递的数据加以处理后交给表示层，提供接口给 Controller 使用。

Controller：表示层，调用 Service 完成业务逻辑并展示给用户。

（2）前端框架 vue+layui

VUE 是用户界面的渐进式框架。可以用来构建用户使用的界面并将视图和模型进行双向绑定。动态更新数据，网页内容随之刷新。

Layui 是一套开源的 Web UI 解决方案，本系统大多使用 Layui 完成前端设计。Layui 遵循原生 HTML/CSS/JS 的开发方式，非常容易上手并使用，风格多变且简约。Layui 兼容目前流行的所有浏览器，与 BootStrap 相比，Layui 拥有了更多自身对经典模块化的理解，可维护性强。在 Layui 的官网中详细的开发文档以及多变的界面展示足以支持本系统对前端界面简洁大方简单的需求。

（3）数据库框架 mysql

Mysql 由瑞典 MySQL AB 公司开发隶属于 Oracle 的关系型数据库管理系统。我们使用的是社区版的开源的 MySQL，首先 MySQL 体积小所有运行速度、命令执行的速度很快，其次 MySQL 是开源的，降低了成本，非常适合于小型网站以及大多数普通用户来使用。同时 MySQL 优化了 SQL 查询语句，提高了查询效率，并与其他大型数据库相比，复杂程度低，易上手。最后 MySQL 可移植性强，提供了 TCP/IP、ODBC、JDBC 等多种数据库链接方式，可以在 UNIX、Linux、Windows 等多个操作平台上使用，可靠、可扩展且快速。

2.4 系统架构

B/S 结构（Browser/Server，浏览器/服务器模式），是一种新型结构，在 WEB 浏览器页面开始盛行后，B/S 模式开始流行。利用 web 浏览器展示数据给用户，用户可直接通过浏览器来实现操作，不必下载客户端就能达到其目的。

本系统的设计采用了基于 web 的 B/S 结构。通过 B/S 架构，客户通过浏览器就可以进行数据交互，浏览器则将在后台与服务器、数据库进行数据交换，实现跨平台交互，节约系统处理数据的大量时间。B/S 是对 C/S 的改进技术，B/S 简化客户端，实现客户端零维护。将系统实现的核心功能逻辑和处理方法集中到服务器端，使得前期系统开发简单，后期维护也得以简化，且系统升级时不必使得每一台客户机重新安装，只需要服务器进行修改发布便可达到全部升级，系统扩展很容易。目前来说只要电脑可以上网，B/S 结构可以在任意的操作系统上使用，且不需安装特定的软件浏览器。

回看如今，互联网的发展使得软件系统无时无刻不在更新升级，为了给客户带来更及时的新颖的体验，发布新内容新功能吸引新用户留住老用户成为软件系统的目的所在。而在 C/S 模式中，频繁的更新、下载将无意识中占用客户不必要的内存空间，同时下载、更新时间也会为用户带去烦恼，所以需要频繁更新、测试的软件系统选择 B/S 的模式将会更加合适。B/S 提供的便利可以让用户不再浪费内存去存储不必要的数据，同时开发人员也不需要频繁发布测试系统，B/S 对于工作量以及用户体验都有着显而易见的优势

最后系统总量总是不变的，客户端负担小，服务器负担则大，所以服务器就需要多样化起来。没有跨平台的问题，服务器所在平台就可以随意选择。Mac 、Linux、Windows、UNIX 都可根据服务器所需来选择， Linux 更加安全，Mac 性能好，Windows 更加简单易上手。同时，开发人员也不必学习新的开发平台，节约了时间，提高了效率，且降低了成本。此外，数据库的选择也更加多样，减少数据库购买成本。

2.5 相关技术简介

（1）JavaScript、Ajax、jQuery

JavaScript 诞生于 1995 年，由 Netscape 公司设计实现而成，取名灵感来源于当时与其相合作的 SUN 公司，取名为 JavaScript 与 Java 相类似，但其内核更加接近于 Self、Scheme。JS 是一种解释型的、具有函数优先的轻量型编程语言，它不仅是运用于 Web 页面的脚本语言，同时也可以在许多非浏览器环境中进行使用。JavaScript 是基于对象的，书写风格简单，支持动态性，可跨平台使用，配合 AJAX 使用，使得界面可以不重载便可快速与用户互动。

Ajax（Asynchronous JavaScript and XML），2005 年诞生，用于描述 HTML、XHTML、CSS、JavaScript/DOM、XML、XSLT、XMLHttpRequest 等技术的新方法。结合多项技术，开发人员使用更加方便快捷，用户可创建出更加高可用、丰富、动态的 Web 用户界面。

jQuery 发布于 2006 年，是一款高速、简介的 JavaScript 框架，它具有链式语法、多功能接口、CSS 选择器等高速特性，方便用户处理 HTML documents、事件以及实现动画效果，与 ajax 的交互也很方便。

JavaScript 是脚本语言，Ajax 则是基于 JS 的组合各项技术的新技术，jQuery 则是 JS 的框架，基于 JS 语言并集合 Ajax 开发出来的库，封装了相关功能，简化操作。

（2）SpringBoot+MyBatis

从 JDBC 到 Spring 到 Spring+MyBatis 最后到 SpringBoot+MyBatis 这是一个连续的过程，并且再一步一步的完善与简便，JDBC 方法需要手动建立数据库连接再执行 SQL，该方法需要频繁的开关数据库链接，同时因为 SQL 与 Java 代码耦合度高，修改时需要两边同时修改，工作量大幅度上升，所以通过 MyBatis 对 SQL 进行封装再 xml 文件中，大大简化了操作，再跟随着 Spring 的发展，两者均逐渐变得简介而有效，直到 SpringBoot，许多类已不再需要手动创建，同时这样的结构也更加专注于业务逻辑以及 SQL 语句的书写。这样的结构使得代码完成效率更高，过程也更加注重业务逻辑重点，不再纠结于配置问题，高效又简洁是其一大特点。

（3） Websocket 即时聊天

Websocket 于 2008 年诞生，其发展于 HTTP Upgrade 协议，是一种可在单个 TCP 连接上进行全双工通信的通信协议[12]，与 HTTP 协议最大的区别主要在于，Websocket 可以做到服务端主动将消息发送给客户端，并建立持久性的建立进行双向的数据传输。这项技术的诞生使得客户端与服务端联系进一步快捷且方便。

第 3 章　系统可行性与需求分析

3.1　系统基本情况描述

随着人们生活水平，经济实力的不断提高，我们买的东西也会越多而杂，很多东西也许使用价值不高，仅仅只是因为冲动购物或喜欢，所以闲置也会越来越多，又因为东西完好又舍不得丢弃，最终导致囤物越来越多，占据大部分生活空间。而这时候二手交易市场的出现就可以解决闲置的问题。

利用二手交易平台，我们可以将自己的不需要的二手物品上架到自己的小店，有需要的人便会来购买，同时如果我们需要某件物品，也可以在平台上用低廉实惠的价格买到心仪的产品，这样闲置的二手商品们得到了二次利用，不再造成资源浪费，保护了环境，商品也得到了有效的利用。

本系统是一款网页版的面向社会的二手交易系统。旨在为大家设计出一款健全的、高效的、简洁的、功能齐全且用户体验良好的“易购”二手交易平台。该系统用户可以通过账号登陆后发布自己身边的闲置，也可以查看与购买其他用户的二手商品，通过实时聊天界面与其他用户在线交流，在个人中心可处理我买到的、我卖出的订单等一系列二手交易工作。同时后台系统工作人员可以管理平台用户，审核用户上架商品，对网站公告进行管理，处理纠纷，同时对网站相关数据进行分析。

3.2　系统可行性分析

系统在开发阶段往往能做到的、能预想到的问题是片面的，当系统投入市场进行运营之后，由于社会环境的变化，人员基数等不定性多变性，系统将逐步面临无法挽回的错误，所以，可行性分析即是在系统投入市场之前，有预见性的、公正的、可靠的、科学性的对系统的主要内容、配套条件进行多方面的进行调查研究并比较分析，最后对系统后续开发与投资提出建设性的意见。当然，世上没有完美“预言”，我们只能尽可能从多个角度多个方向详细的、精准的进行可行性分析。

（1）经济可行性

经济可行性是指可使用资源的可能性。首先从开发系统成本开始。本系统开发软件使用的是社区版的 IntelliJ IDEA 和 MYSQL, 两款软件均为开源且免费使用。紧接着硬件方面使用的都是小组成员普通的能连接网络的电脑，关于服务器我们选择租用的是优惠的学生版经济实惠服务器，所以在开发成本上投入较低。

当系统上线后，系统旨在为用户服务，关于对用户之间相互售卖二手商品的交易是不收取任何中介费用的，而我们系统的经济收益主要来源于用户在首页等显目位置的广告投放。最后系统为社会，为用户将会提供更优质的服务，这点将会产生巨大的社会效益。

（2）技术可行性

本系统选择的开发技术都是目前为止社会上选择较多，性能较高的技术。Java 语言正是目前为止使用最为广泛的网络编程语言之一，其简单稳定可跨平台等等的特点吸引力众多拥护者。MYSQL 拥有开源的社区版服务，这使用很多小型项目都会选择 MYSQL 担任数据库角色，同时 MYSQL 体积小且性能卓越、服务稳定，支持多个操作系统的特点也是其占得市场的优势所在。Layui 一款轻量级框架，易上手且简单美观，非常适合于网页版小型系统的页面设计。最后我们选择的学生款服务器云已经是一个非常成熟、稳定且高效的服务器服务。所以，由此来看，该系统在技术上可行。

（3）操作可行性

从开发人员的角度看，系统采用 SpringBoot+MyBatis 的框架进行开发，将 SQL 语句封装在 MyBatis 中，SpringBoot 架构互相引用且不打扰的分离的框架非常有利于系统的开发。使用的开发工具 IntelliJ IDEA 所自带的自动补充、纠错功能以及调试功能都为开发人员的开发提供了便捷、高效的开发体验。

从用户角度看，用户界面遵循简洁大方的原则，用户只需通过简单的交互便能完成一系列有效操作，界面设计合理，能为用户提供极佳的体验感。

（4）法律可行性

本系统完全遵守国家相关法律法规规定，严格保护用户个人隐私，严令禁止黄暴赌毒出现在该系统中，严格规范用户交易流程，坚决抵制违反国家法律法规的行为。

3.3 系统功能需求分析

3.3.1 系统功能概述

本系统主要包括了卖家模块、买家模块、后台三个模块的设计与实现。

卖家模块主要包含了：商品管理（上架商品、编辑商品、下架商品、查看商品详情）；订单管理（卖家发货、处理买家退货退款申请、查看售出订单、上诉）；用户管理（登录、注册修改个人基本信息、修改密码、修改登陆邮箱）；消息通知。

买家模块主要包含了：商品管理（查询商品、购买商品、评价商品）；订单管理（查看购入订单、买家收货、退货退款申请、上诉）；收藏管理（收藏商品、取消收藏）；地址管理（添加地址、删除地址、修改地址）。

后台模块主要包括了：登录后台系统；用户管理（封禁用户、更改角色）；商品管理（查看详情、审核商品）；公告管理（发布公告、编辑公告、查看详情、删除公告）；纠纷管理（查看详情、处理纠纷）；数据分析；。

3.3.2 系统业务分析

（1）卖家模块

表 3.1 卖家模块功能

功能名称	操作流程	注意事项
登录、注册	用户输入基本信息，后台进行传递到数据库验证，返回登陆（注册）成功。	对密码进行加密处理。注册时对手机号、用户名进行验证。
修改个人信息	用户输入基本信息，后台传递数据进行数据库修改，返回修改成功。	对用户名进行验证。
修改密码	用户输入新旧密码，后台传递数据进行数据库验证并修改，返回修改成功。	对输入的两遍新密码进行验证，后台修改密码时进行加密操作。

续表 3.1

功能名称	操作流程	注意事项
修改邮箱	用户输入新邮箱，验证码，后台传递验证数据并修改数据，返回修改成功。	邮箱验证码的发送与对比，对新邮箱的合法性进行验证。
消息通知	当涉及到相关操作时发送系统消息通知。	
上架商品	用户上传商品基本信息，后台传递验证数据并添加，返回上架成功。	对商品信息的一系列合法约束验证。
下架商品	用户点击按钮，后台修改商品状态。	此可并不完全删除商品，仅修改商品对应状态，后续可重新上架。
编辑商品	用户修改商品信息，后台传递验证数据并修改，返回编辑商品成功。	对商品信息的一系列合法约束验证。
查看订单	用户点击按钮，后台查询数据库后返回订单详细信息。	该查询返回订单详情
发货	用户输入运输单号，后台传递验证数据后更新订单表，返回发货成功。	对订单号的合法性验证。
处理订单申请	用户点击按钮，后台查询相关数据后返回申请详细信息。用户处理，后台传递数据并更新订单状态，返回处理成功。	
上诉	点击上诉按钮，用户输入相关信息，后台传递数据并更新，返回上诉成功。	当卖家买家协议不一致时产生上述。
查看商品详情	用户点击按钮，后台查询数据后返回商品详细信息。	

（2）买家模块

表 3.2 买家模块功能

功能名称	操作流程	注意事项
查询商品	用户输入关键词或选择类别、价格区间或排序、区域等查询方式，后台传递数据，数据库查询相关数据返回。	查询使用模糊查询
购买商品	用户点击购买后选择收货地址，跳转至支付宝支付，支付完成后，后台传递数据更新数据库，返回购买成功。	支付宝沙箱环境，注意订单生成时后台自动填写的订单信息。
评价商品	用户在留言板输入信息后点击按钮，商家可回复。后台传递数据更新数据库，返回评论成功。	留言形式，所有用户均可评论回复。
收货	用户点击收获按钮，后台传递数据更新数据库，返回收获成功。	只有订单状态为已发货才能收货。
退货退款申请	用户点击按钮后，填写相关信息。后台传递数据更新数据库，返回申请成功。	订单未发货之前仅可退款，发货后可退货或仅退款。
上诉	用户点击上诉按钮，后台传递数据更新数据库，返回上诉成功。	当卖家买家双方协议失败时，上诉至后台由工作人员判决。
收藏商品	用户点击按钮，后台传递数据更新数据库。	
取消收藏	用户点击按钮，后台传递数据更新数据库。	
添加地址	用户点击按钮，填写相关信息，后台传递验证数据后添加至数据库，返回添加地址成功。	在个人中心和购买商品两个地方可以添加地址。对地址合理性进行验证。

续表 3.2

功能名称	操作流程	注意事项
删除地址	用户点击按钮，后台传递数据，数据库删除对应数据，返回删除地址成功。	
修改地址	用户点击按钮，填写相关信息，后台传递验证数据后添加至数据库，返回修改地址成功。	仅在个人中心可以修改地址，对修改后的地址合理性进行验证。

（3）后台模块

表 3.3 后台模块功能

功能名称	操作流程	注意事项
登录	用户输入基本信息，后台进行传递到数据库验证，返回登陆（注册）成功。	对密码进行加密处理。
封禁用户	用户点击按钮，后台传递数据后更新数据库。返回封禁成功。	仅修改用户状态，可在封禁成员中对其解封。
更改角色	用户点击按钮，后台传递数据后更新数据库。返回更改成功。	普通成员可设为管理员；管理员可设为后台管理员。三者权力不同。
查看商品	用户点击按钮，后台传递数据后查询数据库。返回该商品详细信息。	该查询返回商品详情。
审核商品	用户点击按钮，后台传递数据后更新数据库。返回审核成功。	审核结果为通过或违规，已上架的商品也可以设置为违规。
发布公告	用户上传公告基本信息，后台传递验证数据并添加，返回发布公告成功。	对公告信息的一系列合法约束验证。

续表 3.3

功能名称	操作流程	注意事项
编辑公告	用户点击修改公告按钮，上传相关信息，后台传递验证数据并修改，返回编辑公告成功。	只有发布公告者才有权力修改其公告。
查看公告	用户点击按钮，后台传递数据后查询数据库。返回该公告详细信息。	该查询返回对应公告详情。
删除公告	用户点击按钮，后台传递数据，数据库删除对应数据，返回删除公告成功。	该操作管理员仅可删除本人发布的公告，后台管理员可以删除全部公告。
查看纠纷	用户点击按钮，后台传递数据后查询数据库。返回该历史纠纷详细信息。	该查询返回所有历史纠纷。
处理纠纷	用户点击按钮，后台传递数据后查询数据库。返回该纠纷详细信息。用户点击判决结果，后台传递数据更新数据库，返回判决成功。	该查询返回对应纠纷详情，管理员可对此进行判决。
数据分析	用户点击按钮，后台传递数据后查询数据库。返回数据后由 Echart 图展示数据分析。	此处采用多种数据分析模型，进行全方位系统分析。

3.3.3 用户 UML 用例图

（1）用户 UML

用户主要分为游客、买家、卖家三种身份，不同身份可以进行不同操作。具体如图 3.1 所示。

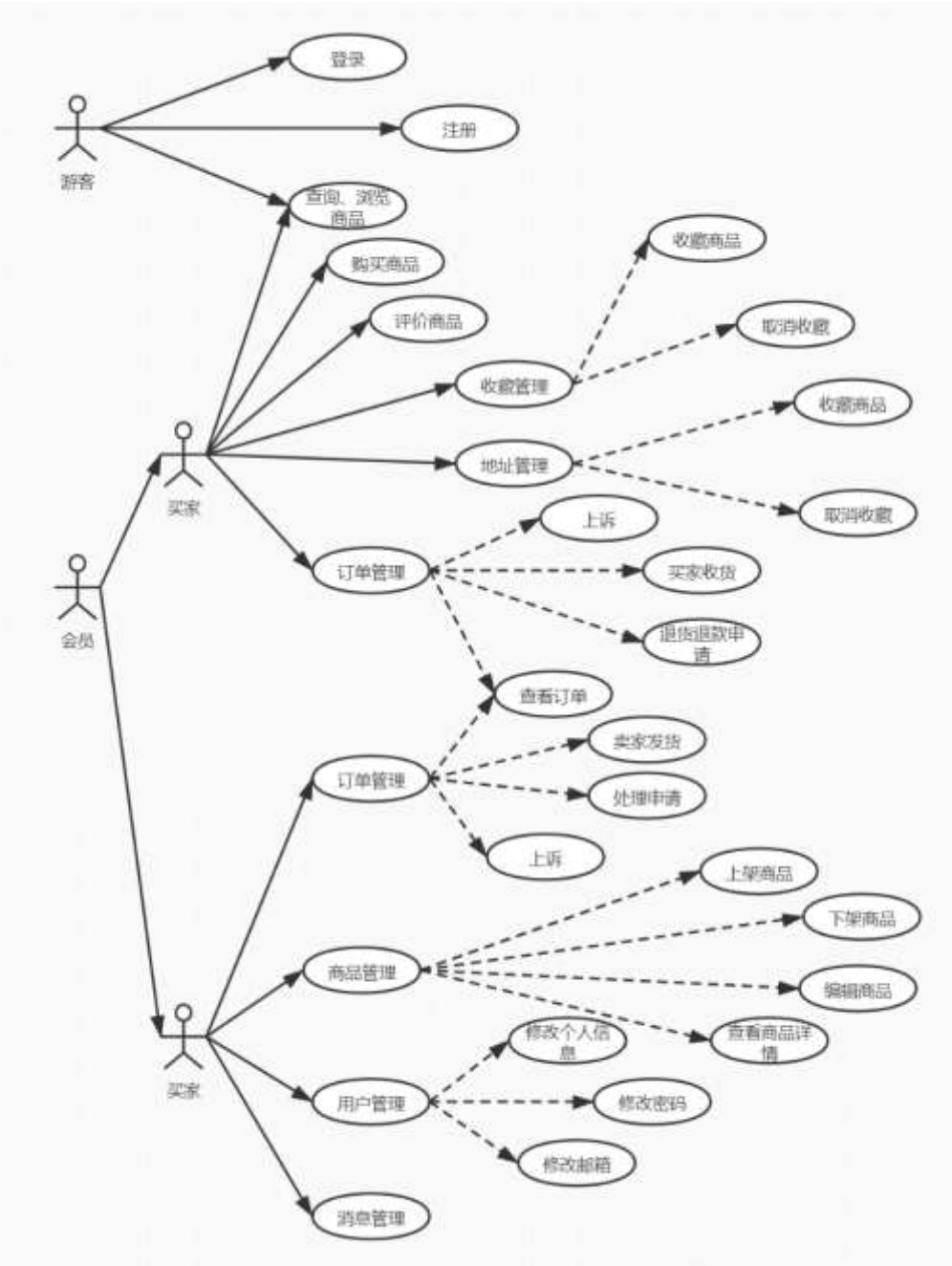

图 3.1 用户 UML 图

（2）管理员 UML

管理员可以对系统整体以及平台等进行管理统计及分析。具体如图 3.2 所示。

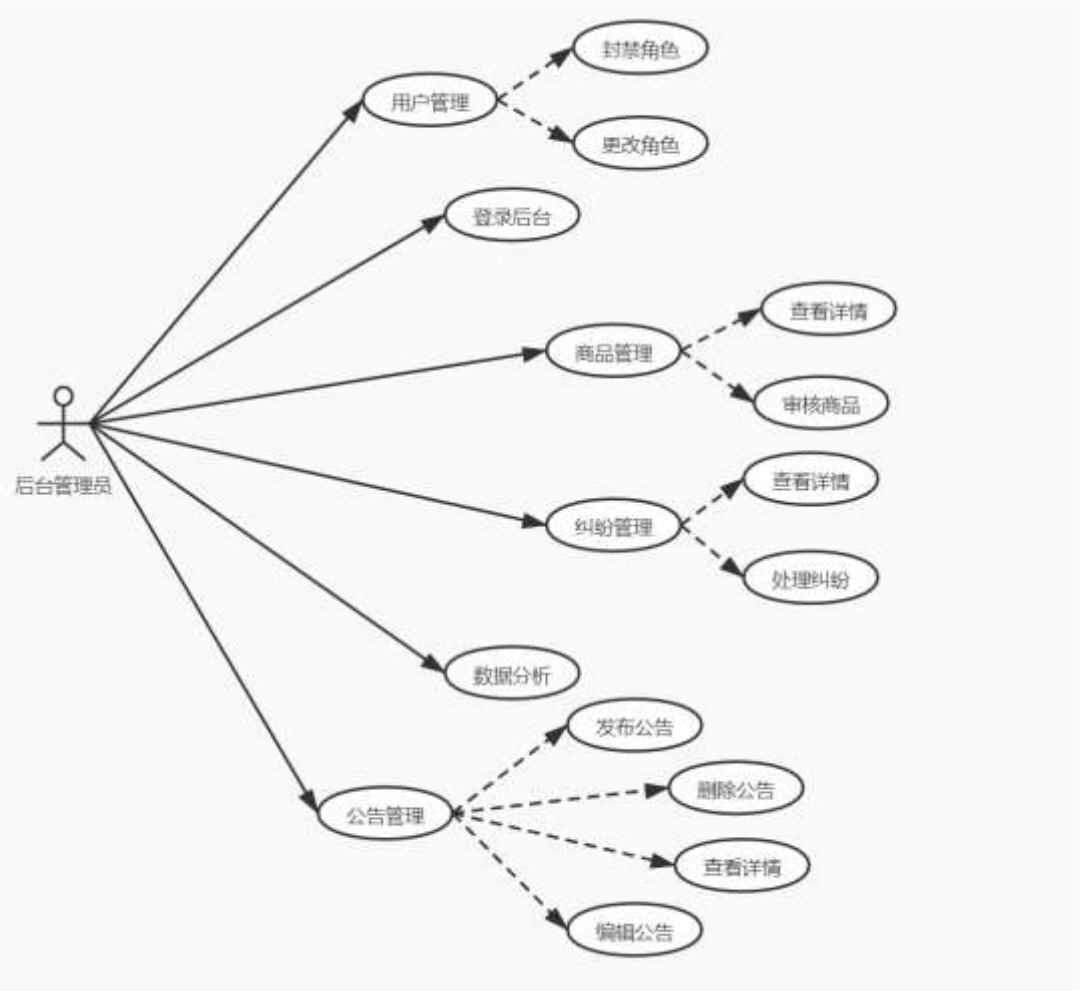

图 3.2 后台 UML 图

第 4 章　系统总体设计

4.1　系统模块设计

系统功能主要包括卖家模块、卖家模块和后台模块。系统总体功能模块如图 4.1 所示。

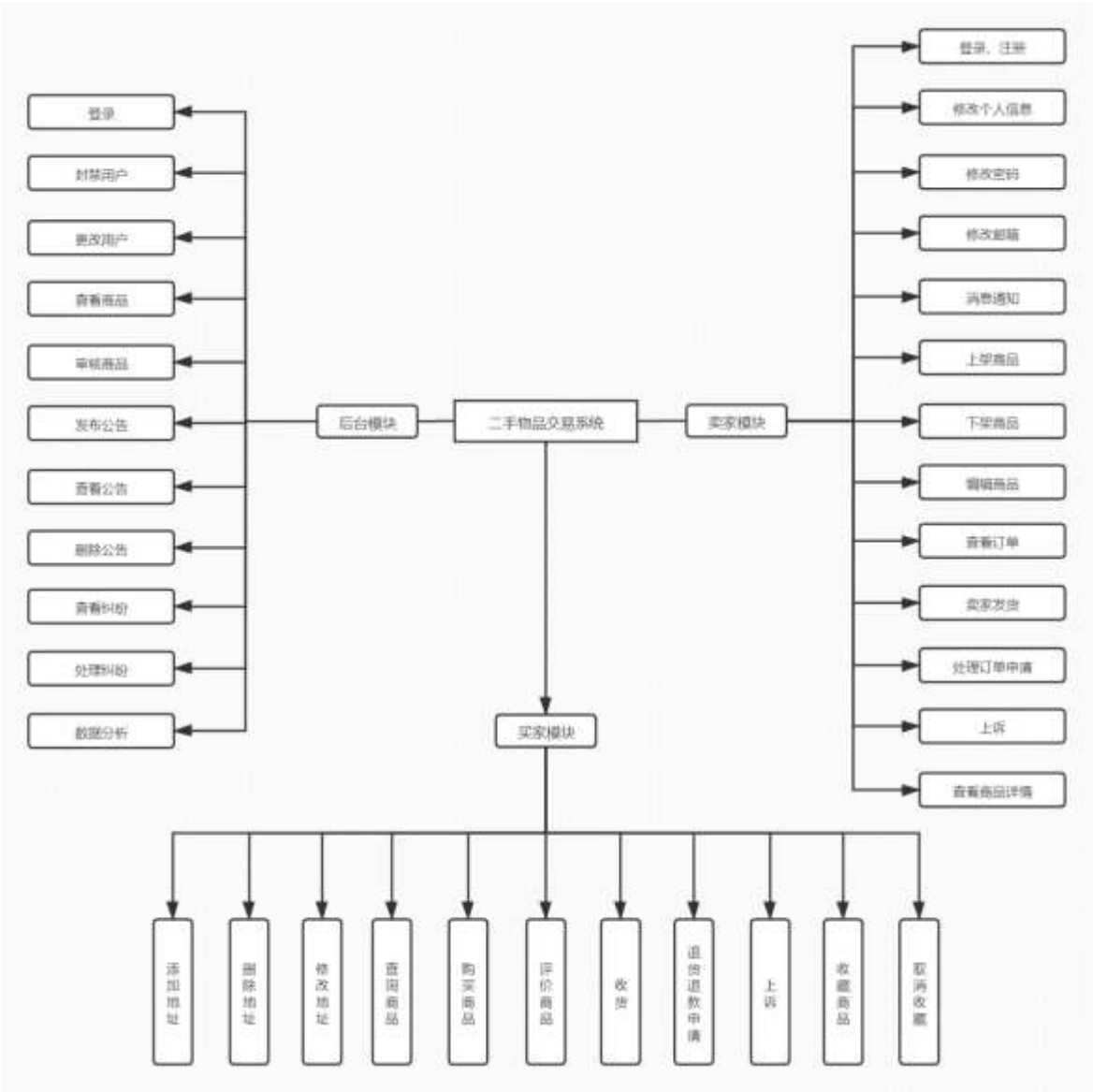

图 4.1 总体功能模块设计

（1） 卖家模块

用户管理：用户注册登陆个人中心后可以对个人信息、密码、邮箱等进行修改。

商品管理：用户可以上架新商品，商品上架后将上传至后台管理员审核，审核通过后即可售卖。已上架商品同时可以由用户下架或由管理员下架，下架商品亦可在编辑修改后重新参与审核并上架。编辑商品即为修改商品已知信息，修改后的商品同样需要后台重新审核。用户也可以点击查看已上架商品详情页面。

订单管理：用户可以查看订单详情，其中包含了买家信息、商品信息以及订单状态等详细信息。针对未发货的订单可以进行发货操作。处理买家的退货退款申请，当意见不统一时可进行上诉交由后台管理员审核处理。

消息通知：当商品审核、商品售出、商品违规、订单申请、订单上诉等情况发生时将由系统发出消息通知。

（2） 买家模块

商品管理：游客与用户均可进行查询商品，浏览主页。用户即可购买商品，选择其收货地址，进行支付宝支付。用户可以对他人商品进行留言，也可以对自己商品留言进行回复。

订单管理：用户可以查看订单详情，其中包含了卖家信息、商品信息以及订单状态等详细信息。针对已发货的订单可以进行收货操作。发起退货退款申请，当意见不统一时可进行上诉交由后台管理员审核处理。

收藏管理：用户可以收藏和取消收藏喜爱的商品。

地址管理：用户可以增加修改删除收货地址，可以将一个地址设置为默认地址。

（3） 后台模块

用户管理：后台管理员登陆后台系统后，可以查看现有系统所有成员，普通成员可以升级为管理员，管理员可以升级为后台管理员。同时后台管理员可以对用户进行封禁与解封处理。

商品管理：管理员可以查看所有商品详情。针对为商家的商品进行审核，审核通过即可上架商品，不通过则发出信息通知卖家修改编辑商品。同时对已上架的商品也可以进行违规下架处理。

公告管理：管理员可以发布修改删除系统公告，其中只有管理员本身才能修改自己的发布的公告，后台管理员也不可以编辑。同时管理员可以删除自己的公告，后台管理员有权利删除所有公告。

纠纷管理：管理员可以处理用户纠纷，也可以查看所有历史纠纷。

数据分析：用户可以查看到系统众多数据集合后的 Echart 图，进行对系统的分析总结。

4.2 系统流程设计

用户进入系统后以游客的身份可以查看商城界面，查找商品。以用户的的身份登录系统，可以出售或购买商品，同时可以对自己的个人信息进行修改，也可以对买到的或卖出的订单进行处理与查看等一系列操作。

普通用户不能进入后台管理系统，以管理员的身份登录管理系统，可以对商品、用户进行管理，也可以对当前系统的一系列数据综合展示进行分析与处理。具体流程图如图 4.2 所示。

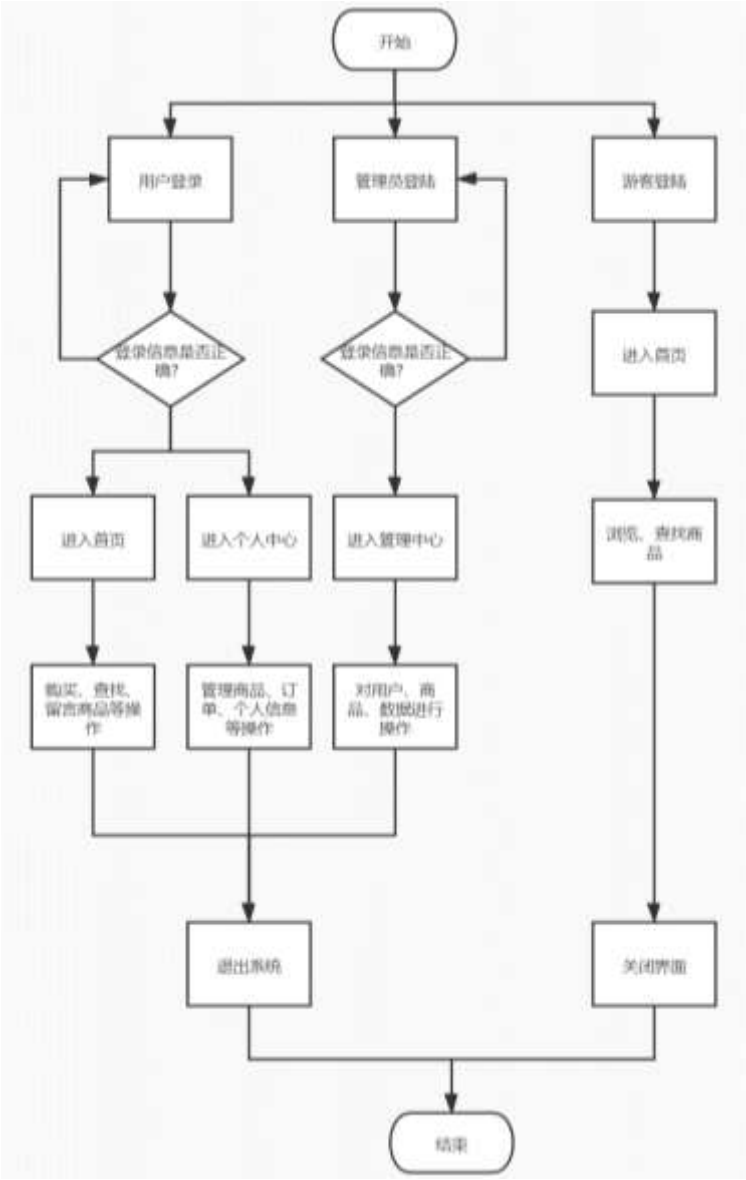

图 4.2 系统流程设计

4.3 系统数据库设计

系统使用 MYSQL 作为存储数据的数据库，设计的数据库表格一共 17 张表格，其中买家模块涉及到其中 8 张表格，分别是登陆表（login）、用户信息表（user_info）、订单纠纷表—（dispute）、订单纠纷图片表（disputeimages）、商品表（commodity）、商品图片表（commimages）、通知消息表（notice）、订单表（soldrecord）。

4.3.1 数据库概念模型

数据库的概念模式是真实世界向机器世界相转化的一个中间层，为了最大陈程度的开发出与人类认识相同的一的软件系统，采用概念模型来对数据库进行建模。本系统的概念模型。

（1）登录

登陆表属性有 id、用户名、邮箱号、用户 id、密码、角色、用户状态。如图 4.3。

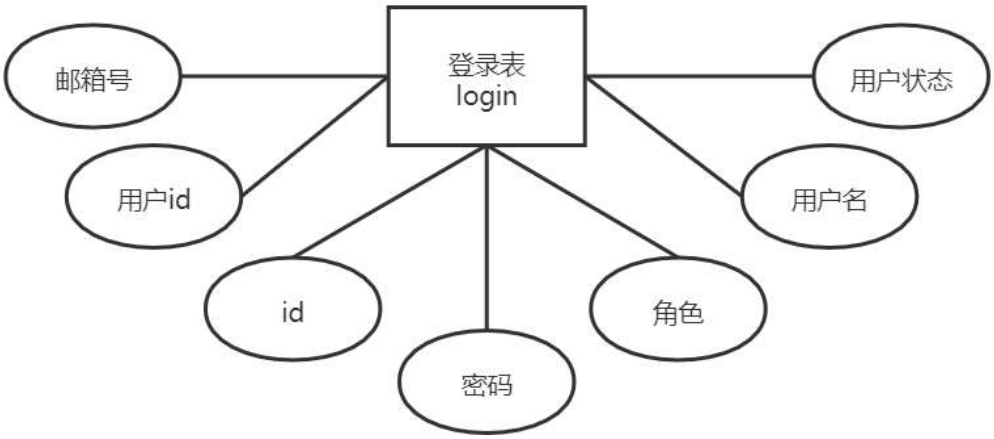

图 4.3 登陆表

（2）用户信息

用户信息表属性有用户 id、用户状态、个性签名、密码、性别、注册时间、账号状态、邮箱、头像、城市、手机号。如图 4.4。

图 4.4 用户信息表

（3）通知

通知表属性有用户 id、通知 id、内容、统治内线、是否为新通知、通知状态、时间。如图 4.5。

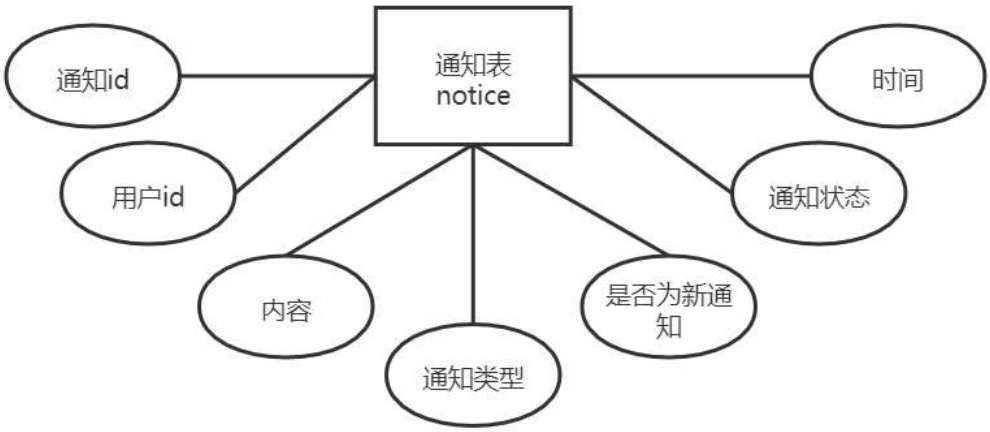

图 4.5 通知表

（4）商品

商品表属性有 id、用户 id、发布时间、修改时间、商品类别、商品类别、简介图、结束时间、商品状态、其他、所在城市、售价、浏览量、原价、视频、商品描述、商品名称。如图 4.6。

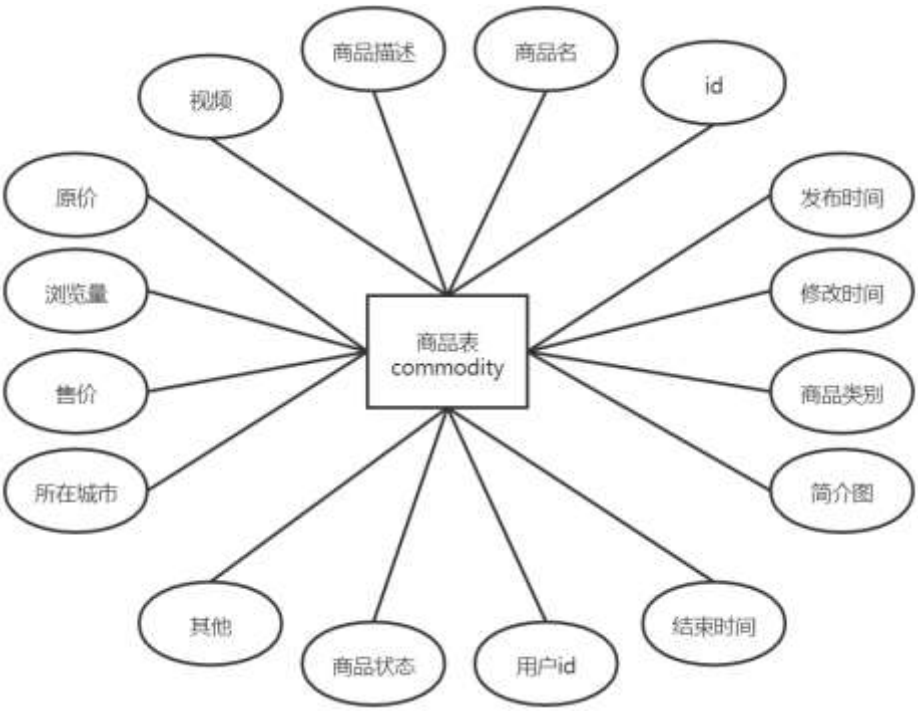

图 4.6 商品表

（5）商品图片

商品图片表属性有 id、商品 id、图片状态、上传时间、图片。如图 4.7。

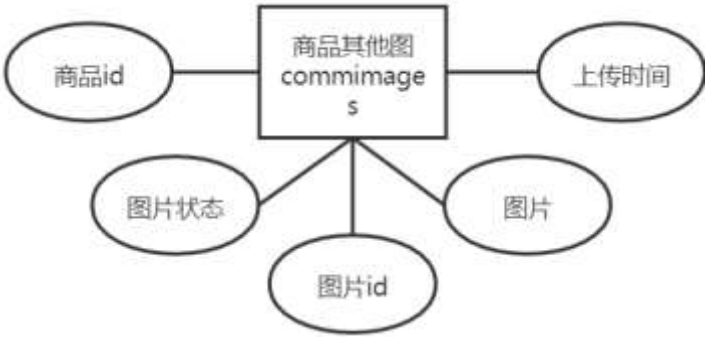

图 4.7 商品图片表

（6）纠纷

纠纷表属性有 id、卖家 id、买家 id、订单 id、买家原因、买家补充、卖家补充、纠纷单状态、订单类型、创建时间、结束时间、纠纷结果、纠纷状态。如图 4.8。

图 4.8 纠纷表

（7）纠纷图片

纠纷图片表属图片 id、订单 id、上传时间、图片、图片所属人。如图 4.9。

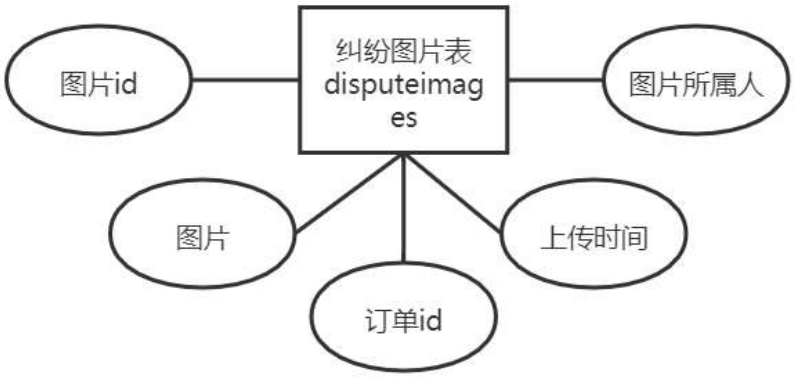

图 4.9 纠纷图片表

（8）订单

订单表属性有 id、卖家 id、买家 id、商品 id、商品名、商品描述、购买时间、售价、订单状态、收货人、联系电话、收货地址、快递编号。如图 4.10。

图 4.10 订单表

4.3.2 数据库逻辑模型

数据库得逻辑模型用于刻画实体之间的关系，建立其中的约束。本系统的逻辑模型具体如图 4.11 所示。

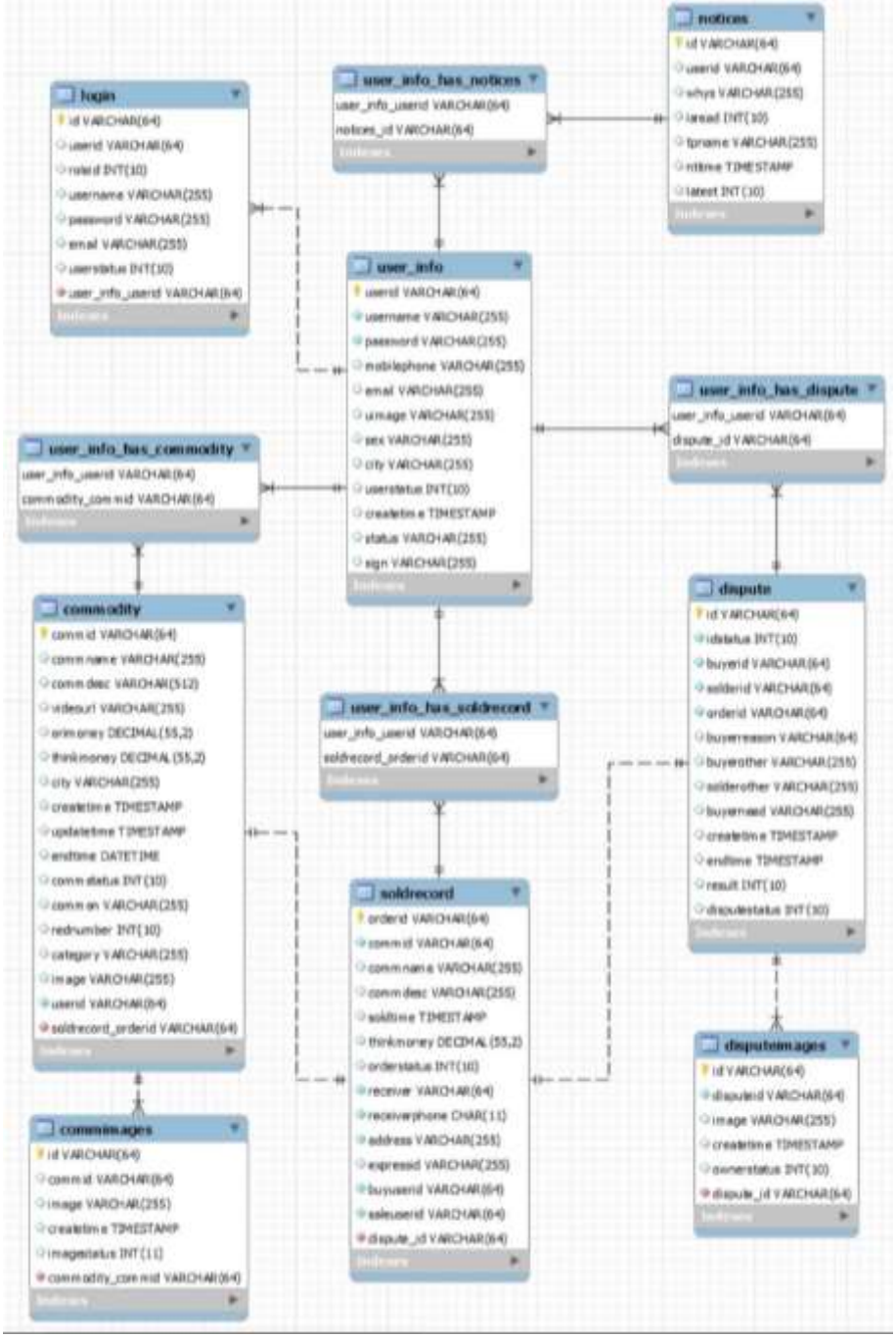

图 4.11 逻辑模型设计

4.3.3 数据库物理模型

表 4.1 用户信息表

字段名称	数据类型	必填	注释
userid	varchar(64)	是	用户 id
username	varchar(255)	是	用户昵称
password	varchar(255)	是	用户密码
mobilephone	varchar(255)	否	手机号
email	varchar(255)	否	用户邮箱
uimage	varchar(255)	否	用户头像
sex	varchar(255)	否	用户性别
city	varchar(255)	否	城市
userstatus	int(10)	否	1 正常 0 封号
createtime	timestamp	否	注册时间
status	varchar(255)	否	用户状态
sign	varchar(255)	否	

表 4.1 user_info 表为用户信息表，主键是 userid，用于保存用户基本信息。

表4.2 纠纷表

字段名称	数据类型	必填	注释
id	varchar(64)	是	id
idstatus	int(10)	是	是纠纷还是退换货 0 纠纷单 1 普通退换单
buyerid	varchar(64)	是	买家 id
solderid	varchar(64)	是	卖家 id
orderid	varchar(64)	是	订单 id
buyerreason	varchar(64)	否	买家原因
buyerother	varchar(255)	否	买家补充
solderother	varchar(255)	否	卖家补充
buyerneed	varchar(255)	否	买家选择的退货、退款
createtime	timestamp	否	创建时间
endtime	timestamp	否	处理结束时间
result	int(10)	否	结果 纠纷：0 买家胜 1 卖家胜 普通：2 同意买家申请 3 拒绝买家申请
disputestatus	int(10)	否	0 历史纠纷单 1 待处理单

表 4.2 dispute 表为纠纷表，主键是 id，用于保存卖家买家退货退款纠纷的详细信息，后期可作为管理员判决凭证。

表 4.3 订单纠纷表

字段名称	数据类型	必填	注释
id	varchar(64)	是	图片 id
disputeid	varchar(64)	是	单子 id
image	varchar(255)	否	图片
createtime	timestamp	否	上传时间
ownerstatus	int(10)	否	0 买家 1 卖家

表 4.3 disputeimages 表为订单纠纷图片表，主键是 id，用于保存纠纷表所上传的图片。

表 4.4 商品表

字段名称	数据类型	必填	注释
commid	varchar(64)	是	商品 id
commname	varchar(255)	否	商品名
commdesc	varchar(512)	否	商品描述
videourl	varchar(255)	否	视频
orimoney	decimal(55,2)	否	原价
thinkmoney	decimal(55,2)	否	售价
city	varchar(255)	否	商品所在城市
createtime	timestamp	否	发布时间
updatetime	timestamp	否	修改时间
endtime	datetime	否	结束时间
commstatus	int(10)	否	0 违规 1 正常 2 删除 3 待审核 4 已售出
common	varchar(255)	否	常用选项：自提，可小刀，不议价等选项
rednumber	int(10)	否	浏览量
category	varchar(255)	否	商品类别
image	varchar(255)	否	简介图
userid	varchar(64)	是	用户 id

表 4.4 commodity 表为商品表，主键是 commid，用于保存商品的基本信息。

表 4.5 商品图片表

字段名称	数据类型	必填	注释
id	varchar(64)	是	图片 id
commid	varchar(64)	否	商品 id
image	varchar(255)	否	图片
createtime	timestamp	否	上传时间
imagestatus	int(11)	否	1 正常 2 删除

表 4.5 commimages 表为商品图片表，主键是 id，用于保存商品其他图片。

表 4.6 消息通知表

字段名称	数据类型	必填	注释
id	varchar(64)	是	通知 id
userid	varchar(64)	否	用户 id
whys	varchar(255)	否	通知内容
isread	int(10)	否	是否阅读 0 未阅读 1 已阅读
tpname	varchar(255)	否	通知类型
nttime	timestamp	否	通知时间
latest	int(10)	否	是否为新通知 1 是 2 不是

表 4.6 notice 表为通知消息表，主键是 id，用于保存整个系统中的消息的基本信息。

表 4.7 订单表

字段名称	数据类型	必填	注释
orderid	varchar(64)	是	订单 id
commid	varchar(64)	是	商品 id
commname	varchar(255)	否	商品名
commdesc	varchar(255)	否	商品描述
soldtime	timestamp	否	购买时间
thinkmoney	decimal(55,2)	否	售价
orderstatus	int(10)	否	0 待发货 1 已发货 2 订单完成 3 申请中 4 交易关闭 5 上诉中
receiver	varchar(64)	是	收货人
receiverphone	char(11)	是	联系电话
address	varchar(255)	是	收货地址

续表 4.7

字段名称	数据类型	必填	注释
expressid	varchar(255)	否	快递编号
buyuserid	varchar(64)	是	买家用户 id
saleuserid	varchar(64)	是	卖家用户 id

表 4.7 soldrecord 表为订单表，主键是 orderid，用于保存买卖双方所生成的订单详细信息。

表 4.8 登陆表

字段名称	数据类型	必填	注释
email	varchar(255)	否	邮箱号
id	varchar(64)	是	登录 id
password	varchar(255)	否	用户密码
roleid	int(10)	否	角色 id 1 普通用户 2 管理员 3 超级管理员
userid	varchar(64)	否	用户 id
username	varchar(255)	否	用户名
userstatus	int(10)	否	1 正常 0 封号

表 4.8 login 表为登录表，主键是 id，用于用户的登录信息、角色以及用户状态。

第 5 章 系统详细设计

5.1 买家模块设计

买家模块以买家身份对功能进行设计与实现，基本功能见图 5.1 所示。

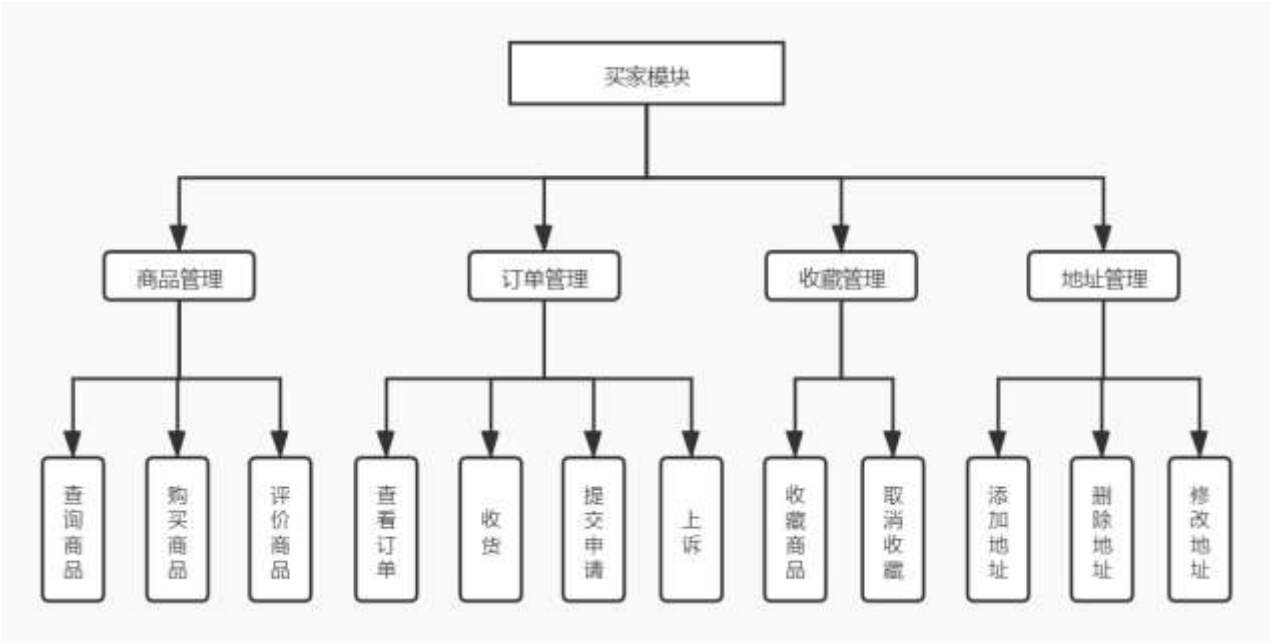

图 5.1 卖家模块设计

（1）商品管理

查询商品：可以通过五种方式进行查询商品，分别是选择类别、规定价格区间、划分区域、价格排序以及关键字查询。用户点击或输入信息后，将提交信息到后端，对数据库查询到的数据进行简单处理后返回到页面进行显示。

购买商品：用户点击心仪的商品，点击购买，选择收货地址信息，也可以在此添加地址。点击确定后将前往支付宝支付，支付成功后，后台将卖家、买家、商品、地址等相关信息封装成对象添加到数据库订单表中，然后返回购买成功的界面。

评价商品：用户在输入留言信息后点击确定，将信息形成表单传至后台，后台封装成留言对象添加至数据库留言表中，然后返回刷新留言界面，显示留言。

（2）订单管理

查看订单详情：在“我买到的”页面点击订单栏的“详情”按钮，前端将订单号等信息传至后端，经过数据库比对查询后返回该订单详情，展示在新弹框中。

买家收货：当检测到订单状态为已发货时，页面操作按钮出现“确认收货”，点击确认收货后，前端将订单号等信息传递至后端，后端修改数据库中订单状态为订单已完成。

退款退货申请：用户点击退款退货按钮，前端获取订单号、商品 id 等信息传递至后端，后端根据前端的传递信息查询并返回商品详情，展示在新弹窗中。用户需要填写新弹窗中的其他信息，点击提交后，前端将获取弹窗所有信息传至后端，由后端进行补充后形成新的纠纷对象，将其加入纠纷表中。同时向卖家发出通知信息。

上诉：当出现卖家不同意退货退款申请时，买家订单页面将出现确认收货和上诉按钮，用户点击确认收货，后端将修改数据库中订单状态为完成订单，同时修改纠纷表状态。用户点击上诉，后端改变纠纷表状态，同时向卖家发起上诉通知。

（3）收藏管理

收藏商品：用户点击收藏商品，前端将判断用户是否登录、收藏商品是否为本人商品，通过检验后由前端传入商品信息，后端补充后将其加入数据库收藏表中。

取消收藏：在个人中心的收藏管理页面中，用户点击“取消收藏”按钮，前端将相关信息传入后端，后端数据库收藏表查询匹配后删除该条收藏记录，返回并刷新界面。

（4）地址管理

添加地址：添加新地址可在购买界面以及地址管理界面进入。点击添加地址，用户输入地址信息，点击确定，前端将判断新地址是否合法，验证通过后将表单信息传至后端，后端补充后，将其添加到数据库地址表中，最后返回并刷新地址信息界面。

删除地址：用户在地址管理界面点击删除地址，前端获取地址信息后传至后端，后端数据库通过匹配查询后执行删除地址操作，最后返回并刷新地址信息界面。

修改地址：用户点击修改地址，前端获取原地址信息展示在弹窗中，用户更新地址信息，点击确定，前端将判断新地址是否合法，验证通过后将表单信息传至后端，后端，由数据库匹配查询后更新该条地址信息，最后返回并刷新地址信息界面。

5.2 后台模块设计

后台模块以管理员身份对功能进行设计与实现，基本功能见图 5.2 所示。

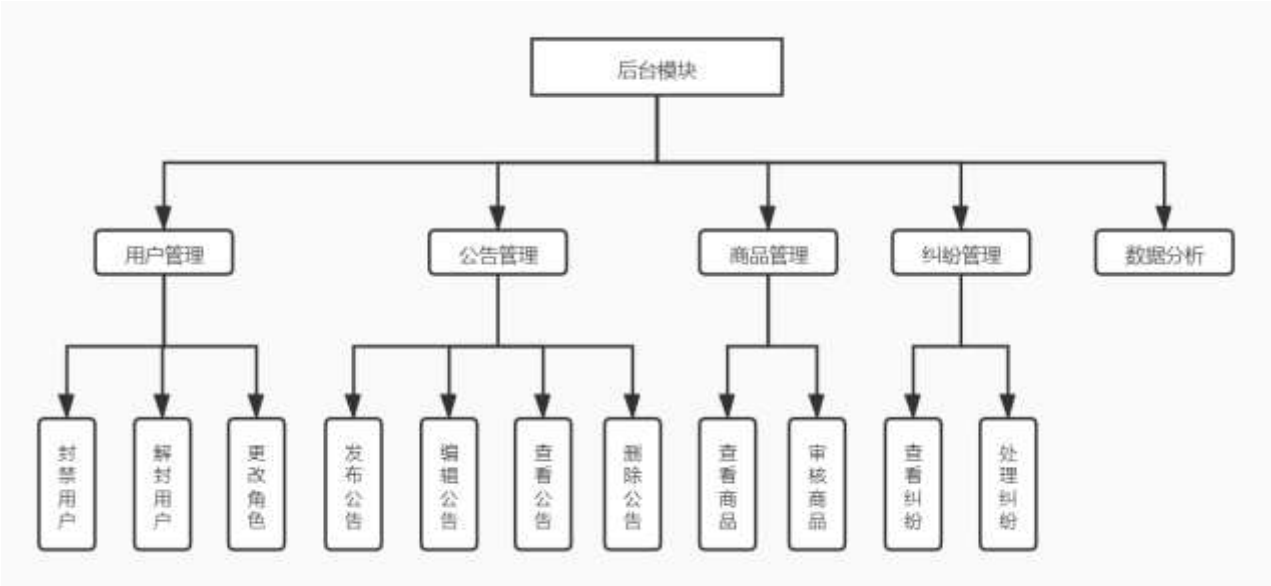

图 5.2 后台模块设计

（1）用户管理

封禁、解封用户：根据用户状态的不同，封禁与普通用户将展示在不同界面。后台管理员点击封禁或解封用户，前端传递用户信息至后端，后端数据库匹配验证后修改用户状态，最后返回并刷新界面。

更改角色：后台管理员有权更改用户角色，普通管理员无权更改。普通用户、管理员、后台管理员将展示在不同界面。后台管理员点击设置用户角色后，前端传递用户信息至后端，后端数据库匹配验证后修改用户角色，最后返回并刷新界面。

（2）商品管理

查看商品详情：管理员点击详情，前端传递商品信息至后端，后端数据库匹配查询该商品状态。状态为正常，后端则返回商品详情页界面；状态为其他，后端则返回商品详情的信息弹窗。

审核商品：未经过审核商品将同时存在“违规”、“通过审核”两种按钮，管理员根据审核结果点击按钮，前端返回商品信息，后端数据库匹配验证后修改商品状态，最后返回并刷新界面。

（3）公告管理

发布公告：管理员填写公告，前端表表单信息传入后端，后端补充了添加至数据库公告表中，返回并刷新界面。

编辑公告：仅有发布者可以编辑公告。发布者修改公告，前端表表单信息传入后端，后端数据库匹配查询后更新公告表，最后返回并刷新界面。

查看公告详情：用户点击详情，前端传入公告信息，后端数据库匹配查询后返回该公告详细信息到弹窗。

删除公告：发布者可以删除自身发布的公告，后台管理员有权删除所有公告。点击删除公告按钮，前端将公告信息传入后端，后端数据库查询匹配后删除该条公告，最后返回并刷新界面。

（4）纠纷管理

查看历史纠纷详情：判决完成后的订单将自动归为于历史纠纷表单中。可进行查看订单和查看结果两个操作。管理员点击订单详情或者查看判决结果，前端传递订单号、纠纷 id 至后端，后端数据库查询匹配后返回该对应全部信息，并展示在页面弹窗中。

处理纠纷：未处理的纠纷将显示在该界面。管理员可进行查看订单和处理纠纷两个操作。管理员点击订单详情，前端传递订单号至后端，后端数据库查询匹配后返回该订单全部信息，并展示在页面弹窗中。管理员点击处理纠纷，前端传递纠纷 id 至后端，后端数据库查询匹配后返回纠纷全部信息并展示在页面弹窗内。接着管理员将根据判决结果点击按钮，前端将其选择传入后端，后端数据库修改该纠纷订单 状态，最后关闭该弹窗。

（5）数据分析

该功能将对售出列表、注册人数与地域关系、注册人数地图表、成交量 Top5 城市偏好进行图形化数据分析。管理员点击对应功能，后端数据库查询匹配统计后返回界面展示。

5.3 卖家模块设计

卖家模块以卖家身份对功能进行设计与实现，基本功能见图 5.3 所示。

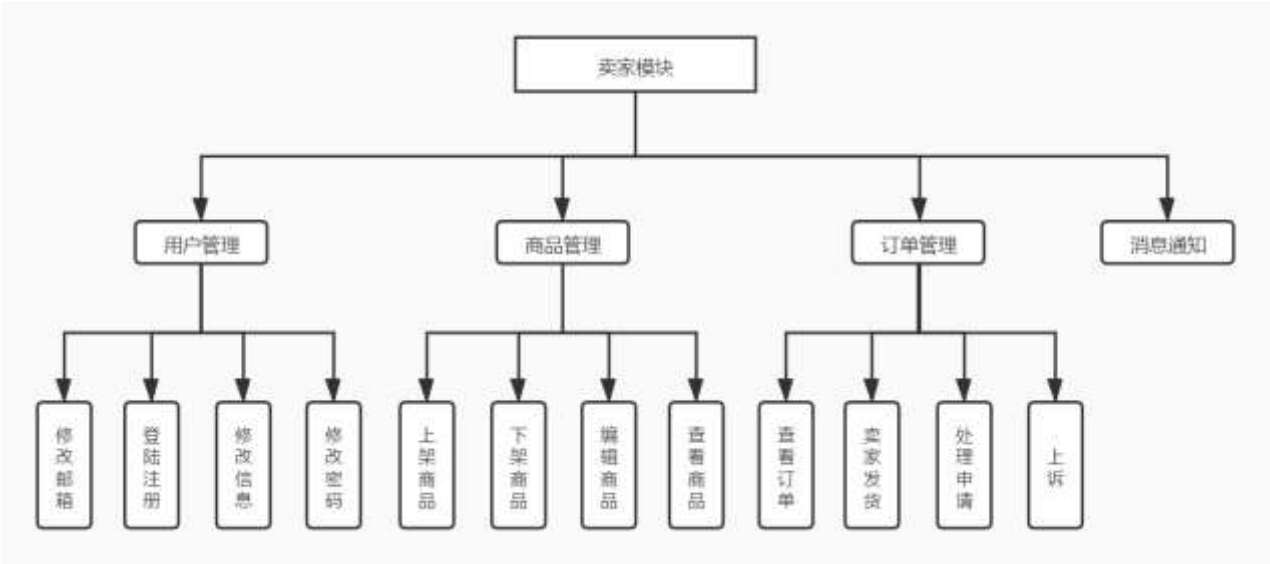

图 5.3 卖家模块设计

（1）用户管理

登陆注册：用户输入相关信息，前端将进行简单的数据合理性检测，验证通过后传至后端，登录信息由后端数据库查询验证通过后返回登陆成功，并刷新界面。注册信息有后端数据库插入新数据进登陆表，接着返回完善个人信息弹窗返回界面。

修改个人信息：用户输入信息，前端将进行简单的数据合理性检测，验证通过后传至后端，后端数据库查询匹配后修改该条数据，最后返回修改成功，并刷新界面。

修改密码：用户输入旧密码和两遍新密码，前端检测两边新密码是否相同，接着传递新旧密码以及用户 id 至后端，后端数据库查询匹配旧密码后，更新登录表密码。最后返回并刷新界面。

修改登录邮箱：用户输入新邮箱，前端简单验证邮箱合理性后将新邮箱、用户 id 传至后端，后端调用邮箱发送验证码服务。此时用户新邮箱将收到验证码，用户输入验证码，前端将验证码传递后端，后端比对成功后，数据查询匹配对应用户信息并修改其邮箱。最后返回更换邮箱成功。

（2）商品管理

上架商品：用户输入信息，前端将进行简单的数据验证，验证通过后传至后端，后端补充数据后形成商品对象，插入到数据库商品表中。新商品需要通过后台管理员的审核，所有此时商品状态为“3 待审核”。

下架商品：用户点击下架商品或管理员点击违规，前端出入商品 id 等信息，后台数据库查询匹配后将商品状态修改为“2 删除”。商品下架后可重新编辑后再次参与审核。

编辑商品：用户点击编辑商品，前端传入商品 id 至后端，后端数据库查询匹配后返回该商品完整信息，展示到弹窗中。用户输入修改信息，前端简单验证其合理性后形成表单传入后端，后端数据库更新该条数据。最后返回修改成功并关闭弹窗，刷新父界面。

查看商品详情：用户点击详情，前端传入商品 id，后端根据商品 id 查询其商品状态，若商品状态为正常，数据库查询匹配其详细信息并返回到新界面中展示。若商品状态为其他，则数据库返回其详细信息至界面弹窗展示。

（3）订单管理

查看订单：在“我卖出的”页面点击订单栏的“详情”按钮，前端将订单号等信息传至后端，经过数据库比对查询后返回该订单详情，展示在新弹框中。

卖家发货：订单状态为“未发货”时将出现发货按钮。用户点击发货，前端展示弹窗，用户输入邮编号点击确定，前端传递订单 id、邮编号至后端，后端数据库查询匹配后更新该条信息。

处理买家申请：用户点击处理申请，前端传入订单 id，后端数据库根据订单 id 查询匹配后返回买家申请信息至界面，卖家根据实际判断结果点击按钮，后端根据其选择修改纠纷订单状态，并发出通知。

上诉：当卖家不同意卖家申请且买家发起上诉时。用户点击处理上诉，输入卖家理由，点击确定，前端简单验证其合理性后形成表单传入后端，后端数据库更新该条纠纷表，并上传至后台系统。最后返回上诉成功，并刷新界面。

（4）消息通知

当用户操作涉及到消息通知后，后端将获取主要信息以及收消息者，同时补充完成后数据库将其添加至通知信息表。

6 卖家模块的实现

6.1 用户管理

6.1.1 修改基本信息

用户在个人中心->个人信息->基本资料中可以修改头像、用户名、手机号、地址、个性签名以及性别。用户在注册后将自动跳转至完善个人信息。界面如图 6.1 所示。

图 6.1 修改信息

（1）头像修改：用户点击头像后，js 将调用修改头像 Controller，接口为'/user/updateuimg'。选择头像后，后端将随机产生图片名，获取文件扩展名，将其拼接在一起后形成文件新名称，上传至本地固定文件夹。接着补充返回对象 res，调用对应 Service 更新用户头像接口，完成更新用户头像，最后返回 res 对象，js 将从 res 属性中判断是否更新成功，code 参数为 0 则上传成功。Controller 如图 6.2。

```
/**
 * 修改头像
 * */
@PostMapping(value = "/user/updateuimg")
@ResponseBody
public JSONObject updateuimg(@RequestParam(value = "file", required = false) MultipartFile file,
HttpSession session) throws IOException {
    JSONObject res = new JSONObject();
    JSONObject resUrl = new JSONObject();
    String filename = UUID.randomUUID().toString().replaceAll("-", "");
    String ext = FilenameUtils.getExtension(file.getOriginalFilename());//获得文件扩展名
    String filenames = filename + "." + ext;//文件全名
    String pathname = "C:\\campusshops\\file\\" + filenames;
   // String pathname = "/softeware/picture/file/" + filenames;
    file.transferTo(new File(pathname));
    resUrl.put("src", "/pic/"+filenames);
    res.put("msg", "");
    res.put("code", 0);
    res.put("data", resUrl);
    String uimgUrl = "/pic/" + filenames;
    String userid=(String) session.getAttribute("userid");
    UserInfo userInfo = new UserInfo().setUserid(userid).setUimage(uimgUrl);
    userInfoService.UpdateUserInfo(userInfo);
    return res;
}
```

图 6.2 修改头像 Controller

（2）其他信息修改：用户输入信息后，前端将会检验手机的合理性，紧接着 js 将其打包成为 JSON 数据，调用修改个人信息接口，接口为"/user/updateinfo"，具体内容如图 6.3 所示。

```
var object = new Object();
    object["username"] = data.field.username;
    object["email"] = data.field.email;
    object["mobilephone"] = data.field.mobilephone;
    //object["city"] = $("#city").val();
    object["city"] = data.field.city;
    object["sign"] = data.field.sign;
    object["sex"] = data.field.sex;;
    var jsonData = JSON.stringify(object);
```

图 6.3 JSON 格式

在 Controller 中，首先获取所有 JSON 数据，首先需要判断用户名是否修改，修改后的用户名是否重复或为空，当两项验证均通过后，调用 Service 更新用户信息。最后将返回一个 ResultVo 对象，其中包含了判断功能的 code。具体内容如图 6.4 所示。

```
@ResponseBody
@PostMapping("/user/updateinfo")
public ResultVo updateInfo(@RequestBody UserInfo userInfo, HttpSession session) {
    String username = userInfo.getUsername();//修改后的用户名
    String email = userInfo.getEmail();//修改后的邮箱
    String userid = (String) session.getAttribute("userid");
    Login login = new Login();//修改后的登录信息
    Login login2 = new Login();//判断是否有重复的登录信息
    Integer integer = 0;//判断返回条数
    if(StringUtils.isEmpty(username)){
        return new ResultVo(false,StatusCode.ERROR,"用户名不能为空"); }
    login.setUserid(userid);
    Login login1 = loginService.userLoginNameIsCopy(login);//根据id查询出的未修改前的数据
    if(login1.getUsername().equals(username) && login1.getEmail().equals(email)){//没有修改名称且邮箱
        userInfo.setUsername(null);
        userInfo.setEmail(null);
    }
    else if(!login1.getUsername().equals(username) && login1.getEmail().equals(email)){//修改了昵称未修改邮箱
        userInfo.setEmail(null);;
        login2.setUsername(username);
        System.out.println(login2);
        integer = loginService.userNameOrEmailExist(login2);
        System.out.println("昵称重复的条数"+integer+"      ");
        if(integer==0) {
            login.setUsername(userInfo.getUsername());
            login.setEmail(userInfo.getEmail());
            System.out.println(login + "    ");
            Integer integer2 = loginService.updateLogin(login);
            System.out.println(integer2 + "     ");
        }
        else return new ResultVo(false, StatusCode.ERROR, "该用户名已存在");
    }
    userInfo.setUserid(userid);
    System.out.println(userInfo);
    Integer integer1 = userInfoService.UpdateUserInfo(userInfo);
    if (integer1 == 1) {
        return new ResultVo(true, StatusCode.OK, "修改成功");
    }
    return new ResultVo(false, StatusCode.ERROR, "修改失败");
```

图 6.4 修改个人信息 Controller

（3）SQL 语句：修改信息采用了不为空就修改的原则，具体语句见图 6.5。

```
<!-- 修改用户信息 -->
  <update id="UpdateUserInfo" parameterType="com.entity.UserInfo">
    update user_info
    <set>
      <if test="username != null">username=#{username},</if>
      <if test="password != null">password=#{password},</if>
      <if test="mobilephone != null">mobilephone=#{mobilephone},</if>
      <if test="email != null">email=#{email},</if>
      <if test="uimage != null">uimage=#{uimage},</if>
      <if test="sex != null">sex=#{sex},</if>
      <if test="city != null">city=#{city},</if>
      <if test="userstatus != null">userstatus=#{userstatus},</if>
      <if test="status != null">status=#{status},</if>
      <if test="sign != null">sign=#{sign}</if>
    </set>
    where userid=#{userid}
  </update>
```

图 6.5 修改个人信息 SQL 语句

（4）具体流程图如图 6.6 所示。

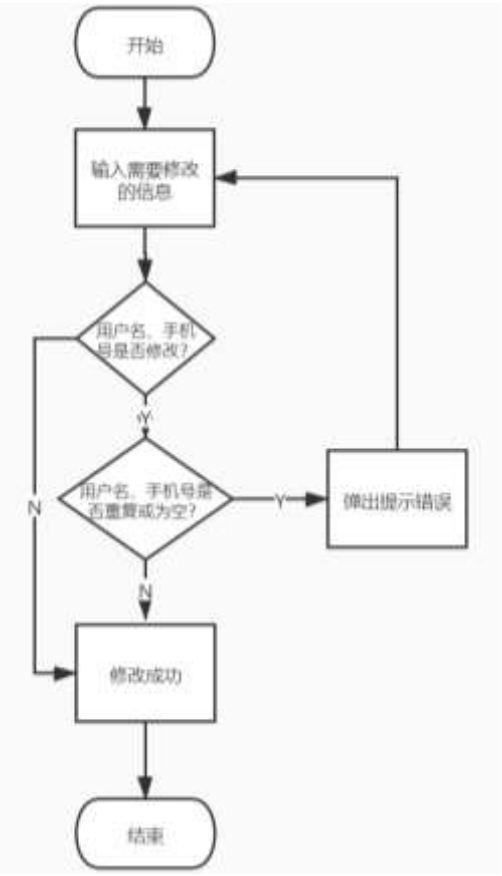

图 6.6 修改信息流程图

6.1.2 修改密码

用户在个人中心->个人信息->修改密码中可以修改密码。用户需要输入当前密码、新密码、确认新密码三项。界面如图 6.7 所示。

图 6.7 修改界面

前端获取用户输入信息后将进行两次验证，首先验证当前新密码格式是否正确，接着验证两次新密码是否输入相同。验证通过后构成 JSON 数据传递调用 Controller 接口，接口名为"/user/updatepwd"。具体如图 6.8 所示。

```
form.on('submit(setmypass)', function (data) {
        var object = new Object();
        object["oldpwd"] = data.field.oldPassword;
        object["newpwd"] = data.field.password;
        if(data.field.password.length<6||data.field.password.length>16){
            layer.msg("密码格式错误", {
                time: 1000,
                icon: 2,
                offset: '100px'
            });
            return false;
        }
        if(data.field.password!=data.field.repassword){
            layer.msg("两次输入的密码不相同", {
                time: 1000,
                icon: 2,
                offset: '100px'
            });
            return false;
        }
        var jsonData = JSON.stringify(object);
        $.ajax({
            url: basePath + "/user/updatepwd",
```

图 6.8 js 判断密码合理性

Controller 中首先获取 JSON 数据，创建新登录 login 对象，填充其用户 id，根据用户 id 调用 Service 查找用户信息函数并补充其 login 对象，紧接着对比当前密码是否输入正确，若正确则进行下一步，将新密码进行加密操作，在调用 Service 函数分别更新登陆表与用户信息表。具体代码如图 6.9 所示，具体流程图如图 6.10 所示。

```
@ResponseBody
@PutMapping("/user/updatepwd")
public ResultVo updatepwd(HttpSession session, HttpServletRequest request) throws IOException {
    JSONObject json = JsonReader.receivePost(request);
    String oldpwd = json.getString("oldpwd");
    String newpwd = json.getString("newpwd");
    String userid = (String) session.getAttribute("userid");
    Login login = new Login();
    UserInfo userInfo = new UserInfo();
    login.setUserid(userid);
    Login login1 = loginService.userLogin(login);
    UserInfo userInfo1 = userInfoService.LookUserinfo(userid);
    String username = userInfo1.getUsername();
    String oldpwds = new Md5Hash(oldpwd, username).toString();
    if (oldpwds.equals(login1.getPassword())){
        //盐加密
        String passwords = new Md5Hash(newpwd, username).toString();
        login.setPassword(passwords);
        userInfo.setPassword(passwords).setUserid(login1.getUserid());
        Integer integer = loginService.updateLogin(login);
        Integer integer1 = userInfoService.UpdateUserInfo(userInfo);
        if (integer == 1 && integer1 == 1) {
            return new ResultVo(true, StatusCode.OK, "修改密码成功");
        }
        return new ResultVo(false, StatusCode.ERROR, "修改密码失败");
    }
    return new ResultVo(false, StatusCode.LOGINERROR, "当前密码错误");
```

图 6.9 修改密码 Controller

图 6.10 修改密码流程图

6.1.3 修改邮箱

用户在个人中心->个人信息->修改邮箱中可以修改邮箱。用户需要输入新邮箱验证码两项。界面如图 6.11 所示。

图 6.11 更换邮箱界面

前端获取用户输入新邮箱和验证码后直接将其传递到 Controller 中，接口名为"/user/updatephone/{email}/{vercode}"。首先验证验证码是否为空、是否正确，若验证码正确，依次修改用户表和登陆表中对应用户的邮箱。具体 Controller 如图 6.12 所示。

```
@ResponseBody
@PutMapping("/user/updatephone/{email}/{vercode}")
public ResultVo updatephone(@PathVariable("email")String email,@PathVariable("vercode")String vercode,HttpSession session) {
    String userid = (String) session.getAttribute("userid");
    String rel = phonecodemap.get(email);
    if (StringUtils.isEmpty(rel)) {//验证码到期 或者 没发送短信验证码
        return new ResultVo(false,StatusCode.ERROR,"请重新获取验证码");
    }
    if (rel.equalsIgnoreCase(vercode)) {//验证码正确
        Login login = new Login().setUserid(userid).setEmail(email);
        UserInfo userInfo = new UserInfo().setUserid(userid).setEmail(email);
        Integer integer = loginService.updateLogin(login);
        Integer integer1 = userInfoService.UpdateUserInfo(userInfo);
        if (integer == 1 && integer1 == 1) {
            return new ResultVo(true, StatusCode.OK, "更换邮箱成功");
        }
        return new ResultVo(false, StatusCode.SERVERERROR, "系统错误，更换失败");
    }
    return new ResultVo(false,StatusCode.ERROR,"验证码错误");
}wd",
```

图 6.12 修改邮箱 Controller

具体流程图如 6.13 所示。

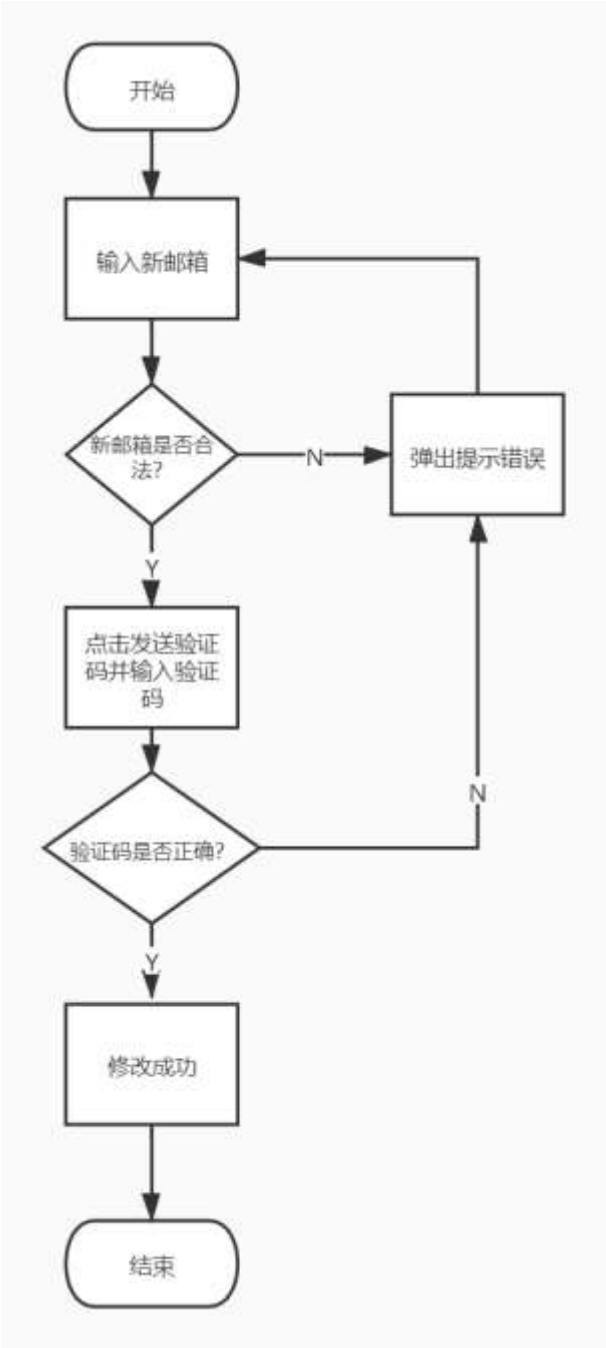

图 6.13 修改邮箱流程图

6.2 商品管理

6.2.1 上架商品

用户在个人中心->商品管理->发布闲置中可以上架商品。用户需要输入各项商品参数。界面如图 6.14 所示。

图 6.14 上架商品界面

（1）其他图上传：上传图片类似于头像修改部分，不同点在于其他图最多可上传三张，所有在 js 中需要判断上传图片的数量，超出则不再允许上传。具体操作如图 6.15 所示。

```
, choose: function (obj) {
    //上传前判断已经上传了多少张图片
    var imgs = document.getElementById("otherimages").getElementsByTagName("img");
    if (imgs.length === 6) {
        layer.msg('最多上传三张图片');
        layer.close(obj);//报错让其停止上传
```

图 6.15 js 判断图片张数

Controller 中方法与头像类似，接口名为'/relgoods/images'，不同点在于由 js 传递过来的数据为列表，在上传图片时需要将图片将其中依次取出仔上传。具体如图 6.16 所示。

```
List<String> imageurls=new ArrayList<>();
for (MultipartFile files:file){
    String filename = UUID.randomUUID().toString().replaceAll("-", "");
    String ext = FilenameUtils.getExtension(files.getOriginalFilename());
    String filenames = filename + "." + ext;
    String pathname = "C:\\campusshops\\file\\" + filenames;
//    String pathname = "/softeware/picture/file/" + filenames;
    files.transferTo(new File(pathname));
    imageurls.add("/pic/"+filenames);
    res.put("msg", "");
    res.put("code", 0);
```

图 6.16 上传其他图片

（2）主图与视频上传：将主图与视频上传结合在一起，利用 js 中 exts 后缀名进行区分，若是主图则 exts: 'png|jpg'，若是视频 exts: 'mp4'。同时视频大小将有所约束，规定视频大小不得大于 20M，判断视频大小的方式是检查 Controller 返回的 res 对象的值。具体 js 如图 6.17 所示。

```
if (res.code > 3) {
    return layer.msg('上传失败');
}else if(res.code > 1){
    return layer.msg('视频大小不能超过20M');
} else {
    layer.closeAll('loading');
    layer.msg('上传成功', {
        time: 1000,
        icon: 1,
        offset: '150px'
```

图 6.17 检查视频大小

Controller 接口为"/relgoods/video"。获取上传视频大小的方法是创建 File 对象 file，利用 file 内置函数 file.length()即可获取到大小，注意这里获得的大小需要除以 1048574 才能转化成 M 为单位。其余上传过程与上类似不再赘述。

（3）商品上传：js 检验商品名称长度，长度不得超过 200。将表单数据封装成 JSON。具体内容如图 6.18 所示。

```
if(data.field.commname.length>200){
    layer.msg('商品名字过长', {
        time: 1000,
        icon: 2,
        offset: '150px'
    });
    return false;
}
var object = new Object();
object["commname"] = data.field.commname;
object["commdesc"] = data.field.commdesc;
object["videourl"] = vuevideo;
object["orimoney"] = data.field.orimoney;
object["thinkmoney"] = data.field.thinkmoney;
object["category"] = data.field.category;
object["common"] = data.field.common;
object["common2"] = data.field.common2;
object["image"] = vuemainimg;
object["otherimg"] = rellistimgs;
var jsonData = JSON.stringify(object);
```

图 6.18 js 传递数据

Controller 接口名为"/relgoods/rel"。首先获取用户 id，调用查找用户信息函数生成 userinfo 对象，随即产生一条商品 id，再利用拼接将所有信息接在一起，调用 Service 插入商品函数上传商品，紧接着将其他图片加入到数据库中。最后条用 Notice 函数发出通知。具体操作如图 6.19 所示。

```
@PostMapping("/relgoods/rel")
@ResponseBody
public String relgoods(@RequestBody Commodity commodity, HttpSession session){
    String userid = (String) session.getAttribute("userid");
    UserInfo userInfo = userInfoService.LookUserinfo(userid);
    String commid = KeyUtil.genUniqueKey();
    commodity.setCommid(commid).setUserid(userid).setCity(userInfo.getCity());//商品id
    commodity.setCommon(commodity.getCommon()+"，"+commodity.getCommon2());//常用选项拼接
    commodityService.InsertCommodity(commodity);
    List<Commimages> commimagesList=new ArrayList<>();
    for (String list:commodity.getOtherimg()) {
        commimagesList.add(new Commimages().setId(KeyUtil.genUniqueKey()).setCommid(commid).setImage(list));
    }
    commimagesService.InsertGoodImages(commimagesList);
    /**发出待审核系统通知*/
    Notices notices = new Notices().setId(KeyUtil.genUniqueKey()).setUserid(userid).setTpname("商品审核")
            .setWhys("您的商品 <a href=/product-detail/"+commid+" style=\"color:#08bf91\" target=\"_blank\"
>"+commodity.getCommname()+"</a> 进入待审核队列，请您耐心等待。");
    noticesService.insertNotices(notices);
    return "0";
```

图 6.19 上传商品 Controller

（4） SQL 语句。插入商品如图 6.20 所示。插入图片如图 6.21 所示

```
<!-- 插入商品 -->
  <insert id="InsertCommodity" parameterType="com.entity.Commodity">
    insert into commodity
(commid,commname,commdesc,videourl,orimoney,thinkmoney,city,common,category,image,userid) value
      (#{commid},#{commname},#{commdesc},#{videourl},#{orimoney},#{thinkmoney},#{city},#{common},#
{category},#{image},#{userid})
  </insert>
```

图 6.20 插入商品 SQL

```
<!-- 插入商品的其他图 -->
<insert id="InsertGoodImages" parameterType="com.entity.Commimages">
  insert into commimages (id, commid, image)
  values
  <foreach collection="list" item="item" index="index" separator=",">
    (#{item.id},#{item.commid},#{item.image})
  </foreach>
</insert>
```

图 6.21 插入商品其他图 SQL

（5）具体流程图如图 6.22 所示。

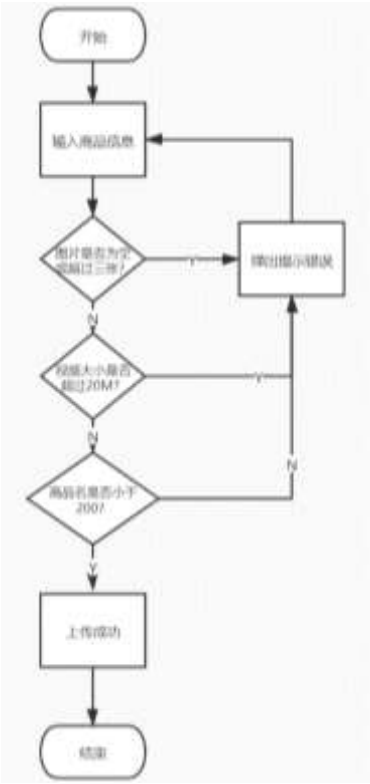

图 6.22 插入商品流程图

6.2.2 编辑商品

用户在个人中心->商品管理->商品清单->编辑中可以编辑商品。用户需要输入各项商品参数。界面如图 6.23 所示。

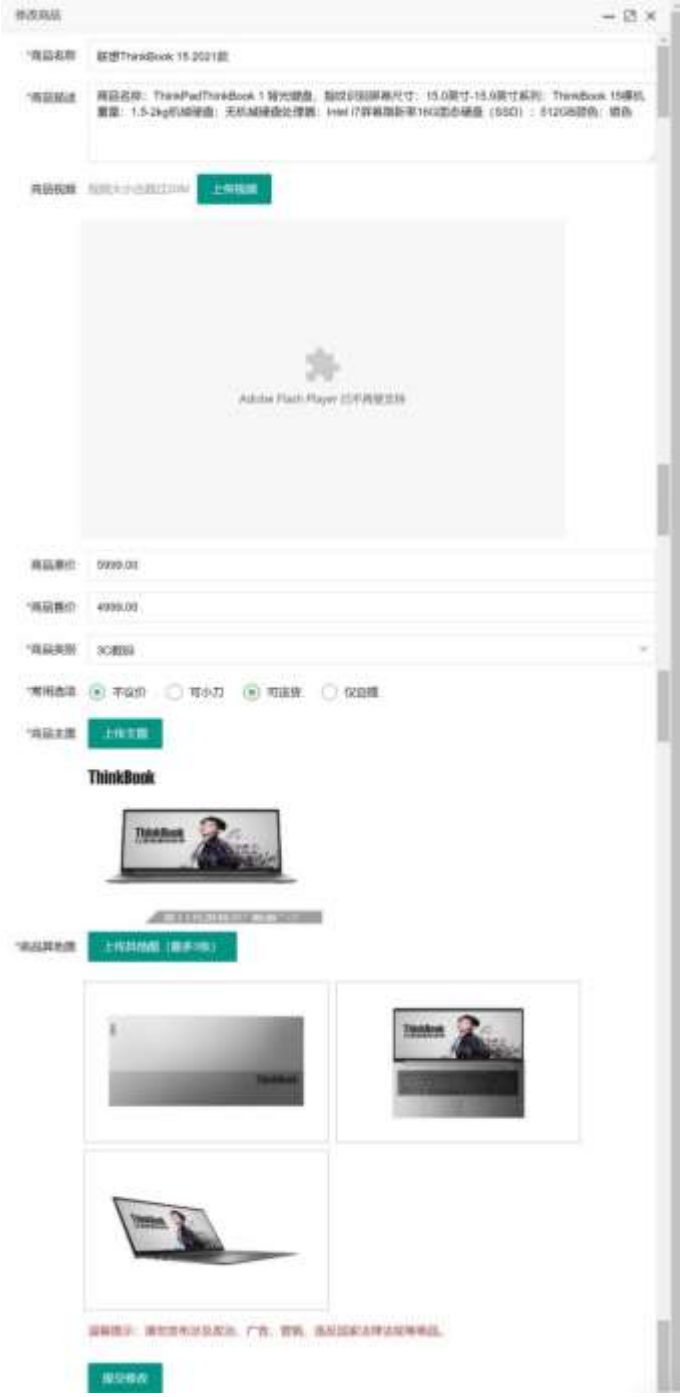

图 6.23 编辑商品界面

首先仔商品清单点击编辑后，js 将商品 id 传递到 Controller 中，接口名为"/user/editgoods/{commid}" 。在这里，后端将会根据商品 id 查询出该商品所有信息，并将其传递至编辑界面，编辑界面名为"user/product/changegoods"。具体如图 6.24 所示。

```
@GetMapping("/user/editgoods/{commid}")
public String toeditgoods(@PathVariable("commid")String commid, HttpSession session, ModelMap modelMap){
    /*String userid = (String)session.getAttribute("userid");
    if(userid==null){
        return "redirect:/";
    }*/
    //根据商品id获取商品信息
    Commodity commodity=commodityService.LookCommodity(new Commodity().setCommid(commid));
    if(commodity.getCommstatus().equals(4)){
        return "error/404";//商品已完成交易
    }
    String[] commons=commodity.getCommon().split("、");
    commodity.setCommon(commons[0]).setCommon2(commons[1]);
    modelMap.put("goods",commodity);
    modelMap.put("otherimg",commimagesService.LookGoodImages(commid));
    return "user/product/changegoods";
}
```

图 6.24 编辑商品 Controller

传递至新界面后操作与上传商品一致，不再赘述。

6.2.3 下架商品

用户在个人中心->商品管理->商品清单->删除中可以删除商品。点击删除按钮后将弹出提示框。界面如图 6.25 所示。

图 6.25 下架商品界面

删除商品并不是直接在数据库中删除该条数据，而是将其商品状态修改为“2 删除”。js 传递商品 id、商品状态值至后端修改商品状态函数，接口名为"/user/changecommstatus/{commid}/{commstatus}"。首先根据商品 id 查询商品状态，若商品状态不为“4 已售出”则可修改其状态，紧接着调用 Service 修改商品状态函数进行修改。具体如图 6.26 所示。

```
@ResponseBody
@GetMapping("/user/changecommstatus/{commid}/{commstatus}")
public ResultVo ChangeCommstatus(@PathVariable("commid") String commid, @PathVariable("commstatus") Integer
commstatus, HttpSession session) {
    String userid = (String) session.getAttribute("userid");
    /**查询售出商品的信息*/
    Integer k;
    Commodity commodity = commodityService.LookCommodity(new Commodity().setCommid(commid));
    if(commodity.getCommstatus() != 4){
        Integer i = commodityService.ChangeCommstatus(commid, commstatus);//返回1则更新成功
        if(commstatus==2)
            k=collectService.updateCollectbyId(commid);

        if(i == 1)
            return new ResultVo(true,StatusCode.OK,"操作成功");
        else
            return new ResultVo(false,StatusCode.ERROR,"操作失败");
    }
    else{
        return new ResultVo(false,StatusCode.ERROR,"该商品已售出，不能删除该商品");
```

图 6.26 下架商品 Controller

下架商品亦可重新上架，用户点击重新上架后将展示与编辑商品相同界面，以下操作均相同，不再赘述。

6.2.4 查看商品

用户在个人中心->商品管理->商品清单中可以查看商品。选择状态即可展示对应商品。该功能可以在界面中查看到商品名称、类别、描述、商品状态、时间以及可进行的操作。点击详情亦可查看商品详细信息。界面如图 6.27 所示。

图 6.27 查看商品界面

（1）选择商品状态：点击下拉框选择商品状态后，js 将传递对应 status 至 Controller，接口名为"/user/commodity/{commstatus}" 。判断传值，若 commstatus 为 100，则默认为展示全部商品，调用查找商品函数进行查找。若 commstatus 不为 100 且有对应值，则传入 commstatus 分类查询并返回展示。具体如图 6.28 所示。

```
@GetMapping("/user/commodity/{commstatus}")
@ResponseBody
public LayuiPageVo userCommodity(@PathVariable("commstatus") Integer commstatus, int limit, int page, HttpSession session) {
    String userid = (String) session.getAttribute("userid");
    //如果未登录，给一个假id
    if(StringUtils.isEmpty(userid)){
        userid = "123456";
    }
    List<Commodity> commodityList=null;
    Integer dataNumber;
    if(commstatus==100){//commstatus未传入值，显示全部商品
        commodityList = commodityService.queryAllCommodity((page - 1) * limit, limit, userid,null);
        dataNumber = commodityService.queryCommodityCount(userid,null);
    }else{//按照传入商品状态类型返回查找商品数据
        commodityList = commodityService.queryAllCommodity((page - 1) * limit, limit, userid,commstatus);
        dataNumber = commodityService.queryCommodityCount(userid,commstatus);
    }
    return new LayuiPageVo("",0,dataNumber,commodityList);
```

图 6.28 根据商品状态查找商品

（2）点击详情：用户点击详情时首先将判断该商品状态是否为 1，若商品状态为 1，则表示商品已上架，此时直接返回商品展示界面。若商品状态不为 1，则表示商品为展示在商城，所以采用弹窗效果展示商品详情。 具体如图 6.29 所示。

```
if (obj.event === 'xiangqing' && data.commstatus === 1) {
    window.open(basePath+"/product-detail/"+data.commid)
}else if(obj.event === 'xiangqing' && data.commstatus != 1){
    console.log("NO");
    layer.open({
        type:2
        ,title:"商品详情"
        ,area:['90%','90%']
        ,content:basePath+'/detailofproduct/'+data.commid
    });
}
```

图 6.29 js 根据商品状态不同处理详情

其中商品展示界面如图 6.30 所示。弹窗界面如图 6.31 所示。

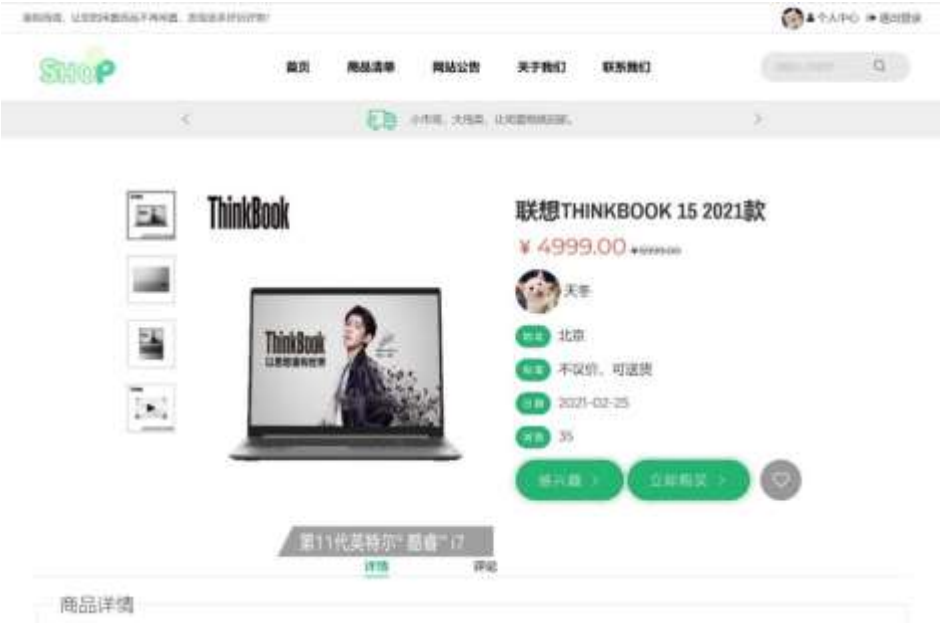

图 6.30 正常商品详情

图 6.31 弹窗界面

弹窗功能 js 将传递商品 id 至 Controller，接口名为"/detailofproduct/{commid}"。根据商品 id 查找出商品的详细信息，再根据商品信息获取其发布者 id，根据 id 查询出用户信息，最后查找出其商品图片。最后展示在弹窗是。具体如图 6.32 所示。

```
@GetMapping("/detailofproduct/{commid}")
public String detailofproduct(@PathVariable("commid") String commid, ModelMap modelMap, HttpSession session){
    Commodity commodity = commodityService.queryCommodityByCommid(commid);
    UserInfo userInfo = userInfoService.LookUserinfo(commodity.getUserid());
    List<String> commimages = commimagesService.LookGoodImages(commodity.getCommid());
    //System.out.println(commodity);
    //System.out.println(userInfo);
    //将数据放入modelmap中
    SimpleDateFormat sdf=new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
    String date = sdf.format(commodity.getCreatetime());
    modelMap.put("detailofproduct",commodity);
    modelMap.put("userinfo",userInfo);
    modelMap.put("time",date);
    modelMap.put("images",commimages);
    return "admin/product/detailofproduct";
```

图 6.32 处理详情 Controller

具体流程图如图 6.33 所示。

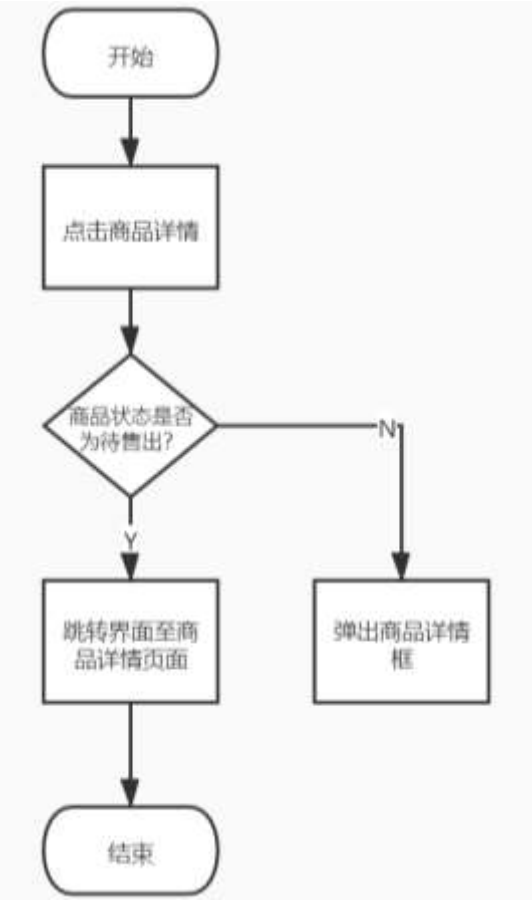

图 6.33 商品详情流程图

6.3 订单管理

6.3.1 查看订单

用户在个人中心->订单管理->我卖出的中可以查看商品。选择状态即可展示对应订单。该功能可以在界面中查看到订单号、名称、描述、售价、售出时间、订单状态、买家 id、快递编号以及可进行的操作。点击详情亦可查看订单详细信息。界面如图 6.34 所示。

图 6.34 订单界面

（1）选择订单状态：此处与商品按选择订单状态显示类似，不同点在于展示信息以及传递的是订单状态。不再赘述。

（2）点击详情：用户点击详情，js 传至商品 id 至 Controller 接口，接口名为"/record-detail/{commid}"。首先根据商品 id 查找出该订单所有信息，在获取当前时间，将其一同放入弹窗界面中，界面名为"user/sold/recorddetail"。界面如图 6.35 所示。

图 6.35 订单详情界面

6.3.2 卖家发货

用户在个人中心->订单管理->我卖出的->发货中可以进行发货。用户需要输入快递编号。界面如图 6.36 所示。

图 6.36 输入快递编号界面

js 获取快递编号后，将快递编号与商品 id 一并传入 Controller 接口中，接口名为"/soldrecord/insertaddress/{expressid}/{commid}"。首先根据商品 id 查询出订单详情，紧接着调用插入地址函数，将地址加入到该订单中。最后发出消息通知。具体操作如图 6.37 所示。

```
@ResponseBody
@PutMapping("/soldrecord/insertaddress/{expressid}/{commid}")
public ResultVo insertAddress(@PathVariable("expressid") String expressid, @PathVariable("commid") String commid, HttpSession session) {
    Soldrecord soldrecord = soldrecordService.record_Detail(commid);
    System.out.println("地址来了" + expressid + "商品编号来了" + commid);
    int orderstatus = 1;
    Integer integer = soldrecordService.InsertAddress(commid, expressid, orderstatus);
    if (integer == 1) {
        Notices notices = new Notices().setId(KeyUtil.genUniqueKey()).setUserid(soldrecord.getBuyuserid()).setTpname("系统通知")
                .setWhys("您的商品 <a href=/product-detail/" + soldrecord.getCommid() + " style=\"color:#08bf91\" target=\"_blank\" >" +
soldrecord.getCommname() + "</a> 已经发货啦。");
        Integer integer1 = noticesService.insertNotices(notices);
        if (integer1 == 1) {
            return new ResultVo(true, StatusCode.OK, "发货成功");
        } else {
            return new ResultVo(false, StatusCode.ERROR, "发货失败");
        }
    }
    return  new ResultVo(false,StatusCode.ERROR,"发货失败");
```

图 6.37 发货 Controller

6.4 退货退款与上诉

（1）买家申请：用户在个人中心->订单管理->我买到的->退款可以进行退货退款。用户需要填写申请信息。界面如图 6.38 所示。

图 3.38 退货退款界面

用户上传信息后将在 js 中判断图片不超过三张，方法同上不再赘述。js 传递 dispute 对象至 Controller 中，接口名为"/dispute/tuikuan"。首先获取传递信息，接着自动生成 id，补充 dispute，最后调用 Service 函数，数据库纠纷表插入一条新数据，同样的方法取出图片，加入图片进数据库，最后发出消息通知。具体如图 3.39 所示。

```
@PostMapping("/dispute/tuikuan")
@ResponseBody
public String tuikuan(@RequestBody Dispute dispute, HttpSession session){
    String orderid = dispute.getOrderid();
    String buyerreason = dispute.getBuyerreason();
    String buyerother = dispute.getBuyerother();
    Soldrecord soldrecord = soldrecordService.queryByOrderid(orderid);
    String id = KeyUtil.genUniqueKey();
    dispute.setId(id).setBuyerother(buyerother).setBuyerreason(buyerreason).setBuyerid(soldrecord.getBuyuserid())
            .setSolderid(soldrecord.getSaleuserid()).setIdstatus("1");
    Integer integer1 = disputeService.Insertdispute(dispute);
    List<Disputeimages> disputeimagesList=new ArrayList<>();
    for (String list:dispute.getOtherimg()) {
        disputeimagesList.add(new Disputeimages() .setId(KeyUtil.genUniqueKey()).setDisputeid(id).setImage(list)); }
    if (!disputeimagesList.isEmpty()){
        disputeimagesService.InsertImages(disputeimagesList); }
    Integer orderstatus = 3;
    Integer integer = buyrecordService.UpdateStatus(orderid,orderstatus);
    if(integer==1 && integer1 == 1) {
        /**发出待审核系统通知*/
        Notices notices = new Notices().setId(KeyUtil.genUniqueKey()).setUserid(soldrecord.getBuyuserid()).setTpname("退款申请")
                .setWhys("您的订单" + soldrecord.getOrderid() + "：" + soldrecord.getCommname() + "</a> 已申请退款，请您耐心等待。");
        noticesService.insertNotices(notices);
        Notices notices1 = new Notices().setId(KeyUtil.genUniqueKey()).setUserid(soldrecord.getSaleuserid()).setTpname("退款申请")
                .setWhys("订单："+soldrecord.getOrderid() + "：" + soldrecord.getCommname() + "</a> 收到退款申请，请您及时处理。");
        noticesService.insertNotices(notices1); }
    return "0";
```

图 3.39 退款申请 Controller

（2）卖家处理：用户在个人中心->订单管理->我卖出的->处理申请中可以进行处理买家爱的申请。用户点击处理申请后，js 将传递订单号至后端，后端数据库匹配查询后将买家太难写的申请信息展示在界面中，卖家根据实际情况点击结果，后端修改纠纷单状态，具体如图 3.40 所示。

```
@PostMapping(value = "/solddispute/{id}/{deal}")
@ResponseBody
public ResultVo SoldDispute(@PathVariable("id") String id,@PathVariable("deal") String deal){
    Dispute dispute = disputeService.queryDisputeById(id);
    Integer check = buyrecordService.UpdateStatus(dispute.getOrderid(),4);
    if("1".equals(deal))
        dispute.setResult(3);
    else
        dispute.setResult(2);
    Integer integer = disputeService.updateDispute(dispute);
    Notices notices = new Notices().setId(KeyUtil.genUniqueKey()).setUserid(dispute.getBuyerid()).setTpname("系统通知")
            .setWhys("您的订单申请"+dispute.getOrderid()+"商家已处理，请及时查收");
    Integer integer1=noticesService.insertNotices(notices);
    if (integer == 1 && integer1 ==1 && check ==1) {
        return new ResultVo(true, StatusCode.OK, "完成");
    }
    return new ResultVo(false, StatusCode.ERROR, "失败");
```

图 3.40 卖家处理 Controller

第 7 章 系统测试

7.1 系统测试目的与方法

（1）系统测试目的

软件测试是软件开发中必要的一项工作，开发人员通过人工测试或者自动化的测试方法[8]来验证软件系统整体的正确性。软件测试的发展是从开始的“调试”，到“证实”“证伪”，再到“评价”，最后到“预防”，已经逐步形成了一整套体系[13]，究其最后的目的是为了对软件进行测试，确认其功能、设计、实现等正确进行，以及发现错误，修改进而改进，让软件质量能进一步得到提升。发展到至今为止，软件测试已成为一门独立的学科，我国也在软件工程领域发表了相应的标准与文件。

（2）常见的系统测试方法

软件测试也有相应的模型，其中改进后的 V 模型相对来说合理[14]。对比传统的 V 模型，改进版的 V 模型将软件测试融入到每各阶段，具体如图 7.1 所示

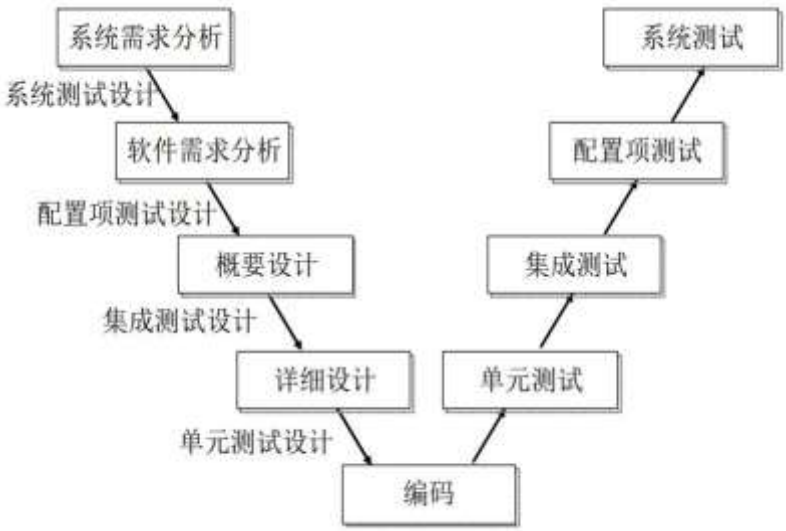

图 7.1 V 模型

①黑盒测试（驱动测试）：假定测试人员在测试软件之前毫不了解软件内部结构，按照实际情况进行多种合法或不合法的行为来测试软件是否正确[15]。这也是日常使用较多的一种测试方法，这样的测试方法有利于我们发现在开发过程中被忽略掉的功能或需求。

②白盒测试[16]：测试人员将对软件整体结构有清楚的认识，在测试将按照事前规定好的步骤进行测试[17]，查看系统是否按照规定进行运转。这样的测试方法有利于提高系统代码的质量。

两种方法虽然有很大的差别，但并非相互对立，非此即彼的，软件测试中常常将两种方法相结合，以达到接近完美的效果。

7.2 系统测试用例

（1）用户管理测试用例

表 7.1 修改基本信息

功能点	前置条件	输入	预期效果
手机不合法	进入个人资料界面	手机号：12321	提示“请输入合法的手机号”
用户名为空	进入个人资料界面	用户名：	提示“用户名不能为空”
用户名重复	进入个人资料界面	用户名：天冬	提示“该用户名已存在”
修改资料成功	输入信息正确	用户名：啾啾 手机号：17320441207 地址：重庆 个性签名：123 性别：选择“女”	修改成功

表 7.2 修改密码

功能点	前置条件	输入	预期效果
当前密码错误	进入修改密码界面	当前密码：000	提示“当前密码错误”
两次新密码不相同	进入修改密码界面	新密码：123123 重复新密码：00	提示“两次输入的密码不相同”
新密码不合法	进入修改密码界面	新密码：000	提示“密码格式错误”
修改密码成功	输入信息正确	当前密码：123456 新密码：000000	修改成功

表 7.3 修改邮箱

功能点	前置条件	输入	预期效果
邮箱、验证码为空	进入修改邮箱界面	邮箱号： 验证码：	提示“请输入邮箱号” 提示“请输入验证码”
邮箱不合法	进入修改邮箱界面	邮箱号：100	提示“邮箱格式不正确”
邮箱重复	进入修改邮箱界面	邮箱号：15231232@qq.com	提示“邮箱号已存在”
验证码错误	进入修改邮箱界面	验证码：000	提示“验证码错误”
修改邮箱成功	输入信息正确	邮箱号：17320441@qq.com 验证码：JKMND	修改成功

（2）商品管理测试用例

表 7.4 上架商品、编辑商品、重新上传

功能点	前置条件	步骤	预期效果
其他图超过三张	已上传三张图片	上传一张新图片	提示“最多上传三张图片”
视频大小超过20M	进入上架商品界面	上传一个视频大小>20M 的视频	提示“视频大小不能超过20M”
无商品主图、其他图	进入上架商品界面	不上传主图、其他图	提示“请上传商品主图” 提示“请上传商品的其他图”
商品名>200	进入上架商品界面	输入 150 字商品名	提示“商品名字过长”
上传商品成功	输入信息正确	输入正确的商品信息	修改成功

表 7.5 下架商品

功能点	前置条件	步骤	预期效果
点击确定	进入下架商品弹窗	点击确定	商品状态修改为“删除”
点击取消	进入下架商品弹窗	点击取消	弹窗关闭

表 7.6 查看商品

功能点	前置条件	步骤	预期效果
选择任意商品状态	进入商品清单界面	点击“待售出”	显示商品状态为“待售出”的商品
查看上架商品	进入商品清单界面	点击待售出商品详情	跳转界面至商品详情
查看未上架商品	进入商品清单界面	点击其他商品详情	弹出商品详情弹出框

（3）订单管理测试用例表

表 7.7 查看订单、发货

功能点	前置条件	步骤	预期效果
选择任意订单状态	进入我卖出的界面	点击“待发货”	显示订单状态为“待发货”的订单
查看订单	进入我卖出的界面	点击订单详情	弹出订单详情弹出框
取消发货	进入发货弹窗	点击取消	弹窗关闭
发货	进入发货弹窗	输入快递编号：10324532 点击确定	提示“发货成功”

7.3 测试总结

作为开发人员总是希望能够全方位、精准的构建软件系统，但在实际开发以及投入正常使用后总会出现问题。例如在开发过程中，由于手机号短信验证目前已关闭不再开源，所以在我们选择了邮箱验证的方法。又例如快递编号的填写，由于对现实生活中快递编号的形成不太了解，所以此处未对快递编号进行太多的检测验证。

第 8 章　结论与展望

本文主要针对二手物品网上交易系统，卖家模块的设计与实现。从二手市场再目前社会上的现状、背景入手，对二手市场国内外市场进行了调研。然后对系统整体架构、框架、开发工具、需求分析、可行性分析进行说明后，开始搭建框架，设计数据库，开始设计。经过小组成员多方面的测试与分析，目前来说系统已经无巨大问题产生，同时在租用服务器的稳定支持下，系统以能够稳定的运行。

经过 3 个月的毕业设计，从一开始的不知从何下手，到回顾就知识，学习新学问，接着是配合小组成员一起设计整体系统，然后开始各种书写板块，再将其合并，最后书写毕业论文。这样的一个过程很完美的解释了什么叫做“万事开头难”，一开始觉得分析一个系统难，后来觉得书写系统代码，最后觉得写一长片篇了论文难。每一个阶段开始时都很“难”，但当我们开始去面对，去解决它时，它又变得“不难”了。

回顾我们的二手物品网上交易系统，这个系统对我们来说并不陌生，早在大三时，我们小组就曾开发过一款类似额图书二手交易商城，但图书二手城只是一个简单的练手，其中还有不少问题被遗留。所以，在进行本次系统开发不仅仅是对图书二手系统的完善，同样也是一次创新，我们不仅优化了整个系统界面风格，重新梳理业务流程，增加系统功能，也争对系统的细节进行了优化与创新。最后，通过本次毕业设计的经历，对于软件功能、二手商城设计以及数据库都有了新的认识。

虽然最后顺利的完成了项目系统的实现，但由于自身能力有限，其中要有需要多细节问解决，对于能投入到社会使用的软件系统还缺乏经验，希望老师批评指正。

致　　谢

参 考 文 献

[1] 2021年中国二手电商市场分析报告-市场规模现状与发展趋势分析[R].观研天下.2021.

[2] 陈倩,张捷.校园二手平台开发与市场分析[J].中国商论,2021,(05):40-41.

[3] 罗定提,张影影,崔杰.基于以旧换新的二手交易市场和捐赠策略研究文献综述[J].中国市场,2021,(13):137-139.

[4]承心如,程诗颢.二手交易平台“闲鱼”评价机制存在问题及其解决对策[J].上海商业,2021,(02):146-147.

[5] 杨松.爱回收“新生”[J].21世纪商业评论,2020,(10):26-29.

[6] 严瑾.把卖二手货变成娱乐[J].成功营销,2016,(10):72.

[7] 胡韩利. 规模将达800亿美元！美国二手市场交易火爆[J/OL]. 全球生态, 2021.

[8] 刘子才. 网络二手市场“柠檬”问题研究[D].四川省社会科学院,2018.

[9] Secondary Market Program--Proposed Regulatory Changes[J]. The Federal Register / FIND,2020,85,(247).

[10] 姚雅峰.Java技术的发展趋势与应用研究[J].无线互联科技,2021,18,(06):81-82.

[11] 陈倩怡,何军.Vue+Springboot+MyBatis技术应用解析[J].电脑编程技巧与维护,2020,(01):14-15+28.

[12] 赵娟娟,刘昌华.WebSocket子协议的设计与实现[J].信息安全研究,2021,7,(01):64-68.

[13] 申晓彦,郭佳旭,曹春芳,杨薇,姚素娟,王霞,邢璐,张晔.基于改进V模型的软件测试过程研究

[14] 丁凝.软件测试过程的质量控制分析[J].网络安全技术与应用,2021,(05):65-66.

[15] 于敏,杨春晖.关于标准中软件测试相关术语的辨析[J].中国标准化,2021,(09):46-54.

[J].电脑知识与技术,2021,17,(09):81-82+87.

[16] Yu Jiujiu,Zhang Jishan,Wu Ning,Mei Yingying,Zhang Deqing,Zhu Lili,Zhu Canglu,Zhu Shaomin. A Functionality Testing Method Based on Exploratory Testing for Android Mobile Apps[J]. Journal of Physics: Conference Series,2021,1856,(1).

[17] Bartenschlager Christina C.,Brunner Jens O.. A new user specific multiple testing method for business applications: The SiMaFlex procedure[J]. Journal of Statistical Planning and Inference,2021,214,(prepublish).